Building Transformers from Scratch

Theory and Applications

A Deep Dive into Large Language Models

Tokenization • Embeddings • Attention • Transformer Blocks

Prof. Anis Koubaa

Alfaisal University

First Edition • 2026

Building Transformers from Scratch: Theory and Applications

First Edition

About the Author

Prof. Anis Koubaa is a professor of Computer Science specializing in Robotics, AI, and Deep Learning. He has extensive experience in both academia and industry, bringing practical insights to theoretical concepts.

About This Book

This textbook provides a comprehensive guide to understanding and building Transformer models from scratch. Starting with the fundamentals of tokenization and embeddings, the book guides readers through the intricacies of self-attention mechanisms, Transformer blocks, and full model architectures.

© 2026 Anis Koubaa. All rights reserved.

License

This work is licensed under the **Creative Commons Attribution-NonCommercial 4.0 International License** (CC BY-NC 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format.
- **Adapt** — remix, transform, and build upon the material.

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to this work, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the author endorses you or your use.
- **NonCommercial** — You may **not** use the material for commercial purposes.

Suggested citation:

Koubaa, A. (2026). *Building Transformers from Scratch: Theory and Applications* (1st ed.). Alfaisal University.

Contact: akoubaa@alfaisal.edu

Contents

1 Introduction to Deep Learning for Language Modeling **4**
1.1 From Artificial Intelligence to Deep Learning . . . 4
1.1.1 A Brief Historical Timeline . . . 4
1.2 Traditional Programming vs Machine Learning . . . 5
1.2.1 The Traditional Programming Paradigm . . . 5
1.2.2 The Machine Learning Paradigm . . . 6
1.3 Data Categories . . . 7
1.3.1 Structured Data . . . 7
1.3.2 Unstructured Data . . . 7
1.4 Learning Paradigms . . . 7
1.4.1 Supervised Learning . . . 8
1.4.2 Unsupervised Learning . . . 8
1.4.3 Reinforcement Learning . . . 9
1.5 Machine Learning vs Deep Learning . . . 9
1.5.1 Feature Extraction . . . 10
1.5.2 Data Dependency . . . 10
1.5.3 Computational Requirements . . . 10
1.5.4 Interpretability . . . 10
1.5.5 Performance Scaling . . . 10
1.6 Deep Learning for Computer Vision . . . 11
1.6.1 How Computers See . . . 11
1.6.2 Convolutional Neural Networks (CNNs) . . . 11
1.6.3 The ImageNet Challenge . . . 13
1.6.4 Computer Vision Tasks . . . 13
1.7 Generative Adversarial Networks (GANs) . . . 13
1.7.1 The Core Idea . . . 14
1.7.2 Mathematical Formulation . . . 14
1.7.3 Applications . . . 15
1.8 The Rise of Natural Language Processing . . . 15
1.8.1 NLP Evolution Timeline . . . 15
1.8.2 Core NLP Tasks . . . 15
1.8.3 Sequential Data and Models . . . 16
1.9 The Transformer Revolution (Preview) . . . 16
1.10 Book Roadmap . . . 17
1.11 Conclusion . . . 18

2 Tokenization for Language Modeling **19**
2.1 Introduction . . . 19
2.1.1 The LLM Pipeline . . . 19
2.2 Mathematical Formalism . . . 20
2.3 From Vocabulary to Indices . . . 21
2.3.1 The Lookup Process . . . 21
2.3.2 Handling Unknown Words (OOV) . . . 21
2.4 Why Tokenization Matters . . . 21
2.5 Evolution of GPT Tokenizers: A Comparative Analysis . . . 22
2.5.1 GPT-2 Tokenizer (50k Vocabulary) . . . 22
2.5.2 GPT-3.5 (Turbo) Tokenizer . . . 22

2.5.3 GPT-4 Tokenizer (cl100k_base) . . . 23
2.6 Unicode and Byte-Level Processing . . . 23
2.6.1 What is Unicode? . . . 23
2.6.2 Why Not Use Unicode Code Points? . . . 25
2.6.3 Why Not Use Raw UTF-8 Bytes? . . . 25
2.7 Tokenization Paradigms . . . 25
2.7.1 1. Character-Level . . . 25
2.7.2 2. Word-Level . . . 25
2.7.3 3. Subword-Level (The Standard) . . . 25
2.8 Algorithm: Byte Pair Encoding (BPE) . . . 26
2.8.1 How It Works . . . 26
2.9 SentencePiece . . . 26
2.10 Special Tokens . . . 27
2.11 Tokenization Performance Metrics . . . 27
2.11.1 Intrinsic Metrics . . . 27
2.11.2 Extrinsic Metrics . . . 29
2.12 Conclusion . . . 29

3 Word Embedding and Vector Representation 30
3.1 The Mathematical Foundation: Vectors . . . 30
3.1.1 Why Everything in AI Must Be a Number . . . 30
3.1.2 Vectors in Two and Three Dimensions . . . 30
3.1.3 Feature Vectors: Ordered Lists of Numerical Attributes . . . 31
3.1.4 Vectors in Computer Vision . . . 31
3.1.5 Vectors in Natural Language Processing . . . 32
3.2 Vector Operations . . . 32
3.2.1 Addition and Subtraction . . . 32
3.2.2 Scalar Multiplication . . . 33
3.3 The Dot Product . . . 33
3.3.1 Properties of the Dot Product . . . 34
3.3.2 Geometric Meaning: Alignment Between Vectors . . . 34
3.4 Vector Normalization . . . 35
3.4.1 Why Normalize? . . . 35
3.4.2 The L2 Norm (Euclidean Norm) . . . 35
3.4.3 The L1 Norm (Manhattan Norm) . . . 36
3.4.4 Comparison of L1 and L2 Norms . . . 36
3.4.5 Implementation in NumPy . . . 36
3.5 Cosine Similarity . . . 37
3.5.1 Why Cosine Similarity Over the Dot Product for NLP . . . 37
3.5.2 Connection to Normalized Vectors . . . 38
3.5.3 Applications of Cosine Similarity . . . 38
3.5.4 Implementation in NumPy . . . 38
3.6 Introduction to Linguistic Embeddings . . . 39
3.6.1 Why Do We Need Embeddings? . . . 39
3.7 One-Hot Encoding and Its Limitations . . . 39
3.7.1 Limitations . . . 39
3.8 Frequency-Based Embedding Models (Count-Based) . . . 40
3.8.1 TF-IDF (Term Frequency-Inverse Document Frequency) . . . 40
3.8.2 Limitations of Count-Based Models . . . 41
3.9 Prediction-Based Embeddings (Neural Probabilistic) . . . 41
3.9.1 Word2Vec Algorithms . . . 41
3.9.2 Technical Deep Dive: The Skip-Gram Model . . . 41

3.9.3 Training Mechanics: How It Actually Learns . . . 42
3.9.4 Linear Substructures (Analogies) . . . 43
3.10 Global Vectors (GloVe) . . . 43
3.10.1 The Core Intuition: Probability Ratios . . . 43
3.10.2 The Mathematical Model . . . 44
3.10.3 Training Algorithm . . . 44
3.11 FastText: Subword Information . . . 45
3.11.1 The n-gram Representation . . . 45
3.11.2 Vector Generation . . . 45
3.11.3 Handling Out-Of-Vocabulary (OOV) Words . . . 45
3.11.4 Example: Arabic Morphology . . . 46
3.12 Conclusion . . . 46

4 From Word Embedding to Contextual Embedding 47
4.1 Introduction — The Limitation of Static Embeddings . . . 47
4.2 Text Pre-Processing for Sequence Models . . . 48
4.2.1 Why Padding is Necessary . . . 48
4.2.2 Masking: Ignoring Padding Tokens . . . 48
4.3 The Perceptron and Feed-Forward Networks (Recap) . . . 50
4.3.1 The Single Perceptron . . . 50
4.3.2 Multi-Layer Perceptron (MLP) . . . 50
4.3.3 The Key Limitation: No Memory . . . 50
4.4 Recurrent Neural Networks (RNN) . . . 50
4.4.1 The Core Idea . . . 51
4.4.2 Mathematical Formulation . . . 51
4.4.3 Worked Example — Sentiment Analysis of “I like AI” . . . 52
4.4.4 How RNNs Create Contextual Embeddings . . . 53
4.4.5 Sequential Model Architectures . . . 54
4.5 Activation Functions . . . 54
4.5.1 The Three Key Activation Functions . . . 55
4.5.2 Comparison . . . 55
4.6 Training RNNs — Backpropagation Through Time (BPTT) . . . 56
4.6.1 The Unrolled Computation Graph . . . 56
4.6.2 The Gradient Chain Rule Through Time . . . 56
4.7 The Vanishing Gradient Problem . . . 57
4.7.1 Mathematical Explanation . . . 57
4.7.2 Intuitive Example . . . 57
4.7.3 Gradient Clipping . . . 58
4.8 LSTM — Long Short-Term Memory . . . 58
4.8.1 The Core Idea . . . 58
4.8.2 The Forget Gate . . . 59
4.8.3 The Input Gate and Candidate . . . 59
4.8.4 The Cell State Update . . . 59
4.8.5 The Output Gate . . . 60
4.8.6 LSTM Variants . . . 61
4.8.7 Limitations of LSTMs . . . 61
4.9 GRU — Gated Recurrent Unit . . . 61
4.9.1 Motivation — Simplifying LSTM . . . 62
4.9.2 The Reset Gate . . . 62
4.9.3 The Update Gate . . . 62
4.9.4 Hidden State Computation . . . 62
4.9.5 LSTM vs GRU Comparison . . . 63

4.10 The Complete Picture 64
4.10.1 The Evolution of Sequence Models 64
4.10.2 Key Equations Reference 64
4.10.3 The Three Fundamental Limitations Motivating Transformers 64
4.11 Conclusion 65

1

Introduction to Deep Learning for Language Modeling

1.1 From Artificial Intelligence to Deep Learning

The quest to build intelligent machines has been one of the most ambitious endeavors in the history of science. Over the past seven decades, this pursuit has evolved through distinct phases—from symbolic reasoning in the 1950s to the data-driven revolution we witness today. To understand where we are headed, we must first understand where we have been.

The field is organized into a clear hierarchy: **Artificial Intelligence** (AI) is the broadest discipline, **Machine Learning** (ML) is a subset of AI, and **Deep Learning** (DL) is a subset of ML. Each layer builds upon the previous one, adding more powerful and more automatic capabilities.

Definition: Artificial Intelligence (AI)

Artificial Intelligence is the science and engineering of creating machines that can perform tasks that would normally require human intelligence. These tasks include reasoning, problem-solving, perception, language understanding, and decision-making. AI encompasses both rule-based (symbolic) systems and learning-based systems.

Definition: Machine Learning (ML)

Machine Learning is a subfield of AI in which systems learn patterns from data rather than being explicitly programmed with rules. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, an ML algorithm finds a function $f : \mathcal{X} \to \mathcal{Y}$ that maps inputs to outputs by minimizing a loss function $\mathcal{L}(f(\mathbf{x}), y)$ over the training data.

Definition: Deep Learning (DL)

Deep Learning is a subfield of Machine Learning that uses artificial neural networks with multiple layers (hence "deep") to automatically learn hierarchical representations of data. A deep neural network computes a composition of nonlinear transformations:

$$f(\mathbf{x}) = f_L(f_{L-1}(\dots f_2(f_1(\mathbf{x}; \theta_1); \theta_2) \dots ; \theta_{L-1}); \theta_L)$$

where each f_ℓ is a layer with learnable parameters θ_ℓ, and L is the depth of the network.

1.1.1 A Brief Historical Timeline

The history of AI is marked by cycles of optimism and disillusionment, often called "AI winters." Understanding this history helps us appreciate why deep learning succeeded where earlier approaches struggled.

- **1950s–1960s: The Birth of AI.** Alan Turing proposed the Turing Test (1950). The Dartmouth Conference (1956) formally established AI as a field. Early systems used hand-coded rules and symbolic logic. The **Perceptron** (Rosenblatt, 1958) was the first neural network model, but it could only learn linearly separable functions.
- **1970s–1980s: Expert Systems and the First AI Winter.** Rule-based expert systems (e.g., MYCIN for medical diagnosis) showed promise but were brittle and expensive to maintain. Minsky and Papert's critique of Perceptrons (1969) dampened interest in neural networks. Funding dried up during the first "AI Winter."
- **1990s–2000s: Machine Learning Flourishes.** Statistical methods replaced symbolic AI. Support Vector Machines (SVMs), Random Forests, and Boosting algorithms achieved strong results on structured data. Neural networks saw a modest revival with **back-propagation** (popularized by Rumelhart, Hinton, and Williams, 1986), but training deep networks remained difficult.
- **2010s–Present: The Deep Learning Revolution.** Three factors converged: (1) massive datasets (ImageNet, Common Crawl), (2) powerful GPUs enabling parallel computation, and (3) algorithmic breakthroughs (ReLU, Dropout, BatchNorm, Transformers). AlexNet (2012) won the ImageNet competition by a wide margin, igniting the modern DL era. By 2017, Transformers redefined NLP, and by 2022, large language models like ChatGPT reached mainstream adoption.

Key Insight:

The Three Pillars of the Deep Learning Revolution Deep learning did not succeed because of a single breakthrough. It required the simultaneous availability of three ingredients: **Big Data** (billions of training examples), **Compute** (GPUs and TPUs capable of trillions of operations per second), and **Algorithms** (architectural innovations like residual connections and self-attention). Remove any one of these pillars, and modern AI would not exist.

1.2 Traditional Programming vs Machine Learning

To understand why machine learning is so powerful, it is essential to contrast it with the traditional programming paradigm that preceded it.

1.2.1 The Traditional Programming Paradigm

In traditional (rule-based) programming, a human expert analyzes the problem, identifies the relevant rules, and encodes them explicitly in code. The program then applies these rules to input data to produce output.

$$\text{Rules} + \text{Data} \xrightarrow{\text{Program}} \text{Output}$$

This approach works well for problems with clear, well-defined logic (e.g., sorting algorithms, tax calculators), but it breaks down when:

- The rules are too numerous to enumerate (e.g., recognizing faces).
- The rules change over time (e.g., spam patterns evolve daily).
- The rules are unknown or difficult to articulate (e.g., "What makes a sentence grammatically natural?").

1.2.2 The Machine Learning Paradigm

Machine learning flips the paradigm. Instead of encoding rules manually, we provide the system with **data** and the corresponding **desired outputs** (labels), and the algorithm **learns the rules** automatically.

$$\text{Data} + \text{Output (Labels)} \xrightarrow{\text{Learning Algorithm}} \text{Rules (Model)}$$

The learned model can then be applied to new, unseen data to produce predictions. The key differences between the two paradigms are:

Dimension	Traditional Programming	Machine Learning
Knowledge Source	Human expert	Data
Rule Creation	Manual	Automatic (learned)
Adaptability	Requires reprogramming	Improves with new data
Scalability	Degrades with complexity	Scales with data
Error Handling	Explicit edge cases	Probabilistic predictions

Table 1.1: Traditional Programming vs Machine Learning

Example:

Spam Classification: Two Paradigms **Traditional Approach:**

```python
def classify_spam(email):
    spam_keywords = ["free", "winner", "click here", "buy now"]
    for keyword in spam_keywords:
        if keyword in email.lower():
            return "SPAM"
    return "NOT SPAM"
```

Problem: Spammers adapt. They write "fr3e" or "cl1ck h3re" to bypass rules. Every new trick requires a new rule.

Machine Learning Approach:

```python
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

# Data + Labels (the "desired outputs")
emails = ["Win a free iPhone!", "Meeting at 3pm", ...]
labels = ["spam", "not_spam", ...]

# The algorithm LEARNS the rules
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)
model = MultinomialNB()
model.fit(X, labels)

# Apply learned rules to new data
prediction = model.predict(vectorizer.transform(["Free gift!"]))
```

Advantage: The model automatically discovers patterns—including obfuscated words, unusual formatting, and sender behavior—without any human rule-writing. It adapts by retraining on new examples.

1.3 Data Categories

Data is the fuel of machine learning. Understanding the types of data is essential because different data types require different modeling strategies. At a high level, data falls into two categories: **structured** and **unstructured**.

1.3.1 Structured Data

Structured data is organized in a predefined format, typically rows and columns. Each column has a fixed data type (integer, float, categorical), and each row represents a single observation.

Examples:

- Relational databases (SQL tables)
- Spreadsheets (Excel, CSV files)
- Sensor readings with timestamps
- Financial transaction records

Traditional ML algorithms—such as Random Forests, Gradient Boosting (XGBoost), and Logistic Regression—excel at structured data. These methods rely on **hand-engineered features** (e.g., "age," "income," "transaction amount") that are already explicitly defined in the columns.

1.3.2 Unstructured Data

Unstructured data has no predefined schema. It is raw, high-dimensional, and requires significant processing before patterns can be extracted.

Examples:

- **Text**: Emails, articles, social media posts, books
- **Images**: Photographs, medical scans, satellite imagery
- **Audio**: Speech recordings, music, podcasts
- **Video**: Surveillance footage, movies, live streams

Deep learning was specifically designed to handle unstructured data. CNNs learn visual features from raw pixels; RNNs and Transformers learn linguistic features from raw text. No manual feature engineering is required.

Key Insight:

The Unstructured Data Explosion According to industry estimates, approximately **80% of enterprise data is unstructured**—text documents, emails, images, and videos that traditional databases cannot easily process. This is precisely the domain where deep learning provides the most value. The ability to automatically extract patterns from unstructured data is what makes DL transformative for industries ranging from healthcare (medical imaging) to finance (document processing) to entertainment (content recommendation).

1.4 Learning Paradigms

Machine learning algorithms are categorized by **how they learn from data**. The three fundamental paradigms are supervised learning, unsupervised learning, and reinforcement learning.

1.4.1 Supervised Learning

In supervised learning, the model learns from a labeled dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $\mathbf{x}_i$ is the input and y_i is the ground-truth label. The goal is to learn a function f that maps inputs to outputs while minimizing a loss function $\mathcal{L}$.

Classification (Discrete Output)

The output y belongs to a finite set of categories $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$.

Examples:

- Email spam detection: $y \in \{\text{spam}, \text{not spam}\}$
- Medical diagnosis: $y \in \{\text{benign}, \text{malignant}\}$
- Handwritten digit recognition: $y \in \{0, 1, 2, \ldots, 9\}$

The standard loss function for classification is **cross-entropy loss**:

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

where y_k is the one-hot encoded true label and $\hat{y}_k$ is the predicted probability for class k.

Regression (Continuous Output)

The output $y \in \mathbb{R}$ is a continuous value.

Examples:

- House price prediction: $y \in \mathbb{R}^+$ (dollars)
- Temperature forecasting: $y \in \mathbb{R}$ (degrees Celsius)
- Stock price prediction: $y \in \mathbb{R}^+$ (closing price)

The standard loss function for regression is the **Mean Squared Error (MSE)**:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Example:

Cross-Entropy Loss Calculation Suppose we have a 3-class classification problem. The true label is class 2, so the one-hot vector is $\mathbf{y} = [0, 1, 0]$. The model predicts probabilities $\hat{\mathbf{y}} = [0.1, 0.7, 0.2]$.
The cross-entropy loss is:

$$\begin{aligned}\mathcal{L}_{\text{CE}} &= -\left(0 \cdot \log(0.1) + 1 \cdot \log(0.7) + 0 \cdot \log(0.2)\right) \\ &= -\log(0.7) \\ &\approx 0.357\end{aligned}$$

If the model were more confident, predicting $\hat{\mathbf{y}} = [0.01, 0.98, 0.01]$, the loss would be $-\log(0.98) \approx 0.020$—much lower, reflecting a better prediction.

1.4.2 Unsupervised Learning

In unsupervised learning, the model receives only inputs $\{\mathbf{x}_i\}_{i=1}^N$ with **no labels**. The goal is to discover hidden structure in the data.

Clustering

Clustering algorithms group similar data points together. The most common algorithm is **K-Means**, which partitions N data points into K clusters by minimizing the within-cluster sum of squares:

$$\arg\min_{\{S_k\}} \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in S_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

where $\boldsymbol{\mu}_k$ is the centroid of cluster S_k.

Applications: Customer segmentation, document grouping, gene expression analysis.

Dimensionality Reduction

High-dimensional data is projected into a lower-dimensional space while preserving important structure.

- **PCA (Principal Component Analysis)**: Finds orthogonal directions of maximum variance. Projects data onto the top d principal components.
- **t-SNE (t-Distributed Stochastic Neighbor Embedding)**: Nonlinear method that preserves local neighborhood structure. Widely used for visualizing high-dimensional embeddings in 2D.

1.4.3 Reinforcement Learning

In reinforcement learning (RL), an **agent** interacts with an **environment** by taking **actions** and receiving **rewards**. The agent learns a **policy** $\pi(a|s)$ that maps states s to actions a to maximize the cumulative reward over time:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

where $\gamma \in [0, 1)$ is the discount factor and r_t is the reward at time step t.

Applications: Game playing (AlphaGo, Atari), robotics, autonomous driving.

Key Insight:

Reinforcement Learning from Human Feedback (RLHF) Modern large language models like ChatGPT are fine-tuned using **RLHF**. After pre-training on text data (unsupervised) and fine-tuning on instruction-following examples (supervised), the model is further refined using reinforcement learning: human evaluators rank model outputs, a reward model is trained on these rankings, and RL (specifically PPO—Proximal Policy Optimization) is used to optimize the language model's policy to produce outputs that humans prefer. This is why ChatGPT feels more "aligned" than raw GPT-3.

1.5 Machine Learning vs Deep Learning

While deep learning is technically a subset of machine learning, the practical differences between traditional ML and DL are significant enough to warrant careful comparison.

1.5.1 Feature Extraction

- **Traditional ML**: Requires **manual feature engineering**. A domain expert must decide which features to extract (e.g., word frequency, edge histograms). The model's performance is bottlenecked by the quality of these features.
- **Deep Learning**: Performs **automatic feature extraction**. The network learns hierarchical representations directly from raw data. Early layers learn low-level features (edges, phonemes), and deeper layers learn high-level abstractions (objects, meaning).

1.5.2 Data Dependency

- **Traditional ML**: Works well with small to medium datasets (hundreds to thousands of samples). Algorithms like SVMs and Random Forests are data-efficient.
- **Deep Learning**: Requires large datasets (thousands to millions of samples). Deep networks have millions of parameters and need massive data to avoid overfitting.

1.5.3 Computational Requirements

- **Traditional ML**: Can be trained on CPUs in minutes to hours.
- **Deep Learning**: Requires GPUs or TPUs. Training can take hours to weeks. Large language models require clusters of thousands of GPUs.

1.5.4 Interpretability

- **Traditional ML**: Models like Decision Trees and Linear Regression are inherently interpretable. You can inspect feature importances and decision boundaries.
- **Deep Learning**: Often described as a "black box." With millions of parameters, it is difficult to understand *why* a particular prediction was made. Active research in Explainable AI (XAI) seeks to address this.

1.5.5 Performance Scaling

- **Traditional ML**: Performance **plateaus** after a certain amount of data. Adding more data yields diminishing returns.
- **Deep Learning**: Performance continues to **scale** with more data and larger models. This is the key insight behind "scaling laws" that drive modern LLM development.

Dimension	Traditional ML	Deep Learning
Feature Extraction	Manual (domain expert)	Automatic (learned)
Data Requirement	Small–Medium	Large–Massive
Compute	CPU (minutes–hours)	GPU/TPU (hours–weeks)
Interpretability	High (transparent)	Low (black box)
Scaling with Data	Plateaus	Continues to improve

Table 1.2: Machine Learning vs Deep Learning: A Comparative Summary

Key Insight:

The Performance–Data Curve If you plot model performance (y-axis) against dataset size (x-axis), traditional ML algorithms rise quickly but flatten early, while deep learning models start slower but continue to climb with more data. The crossover point—where DL surpasses ML—typically occurs at tens of thousands of samples. Below this threshold, traditional

ML often wins because it does not overfit as easily on small data. Above it, deep learning dominates because it can exploit the additional signal that simpler models cannot capture.

1.6 Deep Learning for Computer Vision

Computer vision was the first domain where deep learning achieved superhuman performance, and it remains one of the best illustrations of how deep networks learn hierarchical representations.

1.6.1 How Computers See

To a computer, an image is simply a matrix of numbers. Each number represents the intensity of a pixel.

- **Grayscale Image**: A 2D matrix of shape $H \times W$, where each entry is a pixel intensity in the range $[0, 255]$. A value of 0 is black, and 255 is white.
- **Color Image (RGB)**: A 3D tensor of shape $H \times W \times 3$, where the three channels represent Red, Green, and Blue intensities. Each channel is a grayscale matrix.

Example:

A Grayscale Image as a Matrix Consider a tiny 4×4 grayscale image representing a simple pattern:

$$\mathbf{I} = \begin{bmatrix} 0 & 0 & 255 & 255 \\ 0 & 0 & 255 & 255 \\ 255 & 255 & 0 & 0 \\ 255 & 255 & 0 & 0 \end{bmatrix}$$

This matrix represents a 4×4 image that is black in the top-left and bottom-right quadrants, and white in the top-right and bottom-left quadrants—a simple checkerboard pattern. The computer "sees" only these numbers; it has no inherent concept of shapes or patterns. The job of a neural network is to learn such concepts from data.

1.6.2 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks are the foundational architecture for deep learning in computer vision. Instead of connecting every input pixel to every neuron (which would be computationally prohibitive), CNNs use **local filters** that slide across the image, extracting features at each position.

The Convolution Operation

A convolution applies a small filter (kernel) $\mathbf{K}$ of size $F \times F$ to an input image $\mathbf{I}$ of size $W \times W$. The filter slides over the image with a **stride** S and optional **padding** P, computing element-wise products and summing them at each position.

The output feature map has spatial dimensions:

$$W_{\text{out}} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

Pooling Layers

After convolution, **pooling layers** reduce the spatial dimensions while retaining the most important information:

- **Max Pooling**: Takes the maximum value in each local window. Retains the strongest activations.
- **Average Pooling**: Takes the average value. Provides a smoother summary.

A 2×2 max pooling with stride 2 reduces each spatial dimension by half.

Activation Functions

The **ReLU** (Rectified Linear Unit) activation is the standard nonlinearity in CNNs:

$$\mathrm{ReLU}(x) = \max(0, x)$$

ReLU is computationally efficient and mitigates the vanishing gradient problem that plagued earlier networks using sigmoid or tanh activations.

Feature Hierarchy

A key insight of CNNs is that successive layers learn increasingly abstract features:

1. **Layer 1**: Edges, corners, color gradients
2. **Layer 2**: Textures, simple patterns
3. **Layer 3**: Object parts (eyes, wheels, windows)
4. **Layer 4+**: Entire objects, scenes, semantic concepts

This hierarchical feature learning is what makes deep networks so powerful—they automatically discover the relevant features at each level of abstraction.

Example:

Worked Example: 2D Convolution Consider a 5×5 input matrix $\mathbf{I}$ and a 3×3 filter $\mathbf{K}$ with stride $S = 1$ and no padding ($P = 0$).

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Output size: $W_{\text{out}} = \lfloor (5 - 3 + 0)/1 \rfloor + 1 = 3$. The output is a 3×3 matrix.
Top-left element (position $(0, 0)$): We overlay $\mathbf{K}$ on the top-left 3×3 patch of $\mathbf{I}$:

$$\begin{aligned} \mathrm{Output}(0,0) &= (1 \cdot 1) + (0 \cdot 0) + (1 \cdot (-1)) \\ &\quad + (0 \cdot 1) + (1 \cdot 0) + (0 \cdot (-1)) \\ &\quad + (1 \cdot 1) + (0 \cdot 0) + (1 \cdot (-1)) \\ &= 1 + 0 - 1 + 0 + 0 + 0 + 1 + 0 - 1 = 0 \end{aligned}$$

This filter acts as a **vertical edge detector**: it produces high values when there is a strong contrast between the left and right sides of the receptive field, and zero when the pattern is symmetric.

1.6.3 The ImageNet Challenge

The **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** was the proving ground for deep learning. Each year, teams competed to classify 1.2 million images into 1,000 categories. The results tell a remarkable story:

Year	Model	Depth (Layers)	Top-5 Error (%)
2010	Shallow (hand-crafted)	—	28.2
2012	AlexNet	8	16.4
2014	VGG	19	7.3
2014	GoogLeNet	22	6.7
2015	ResNet	152	3.57
—	Human Performance	—	∼5.1

Table 1.3: ImageNet Challenge Results: The Rise of Deep Learning

The pivotal moment was **2012**, when Alex Krizhevsky's **AlexNet**—a CNN with just 8 layers trained on GPUs—reduced the error rate from 28.2% to 16.4%, a margin of victory that shocked the computer vision community. By 2015, Microsoft's **ResNet** with 152 layers surpassed human-level performance with a 3.57% error rate.

Key Insight:

The Depth Revolution In just three years (2012–2015), the winning models went from 8 layers (AlexNet) to 152 layers (ResNet). This explosion in depth was made possible by **residual connections**—skip connections that allow gradients to flow directly through the network, solving the vanishing gradient problem. Without residual connections, training a 152-layer network would have been impossible. The key innovation of ResNet was not "more layers" but "a way to train more layers."

1.6.4 Computer Vision Tasks

Deep learning has enabled a progression of increasingly sophisticated computer vision tasks:

1. **Image Classification**: Assign a single label to an entire image (e.g., "cat" or "dog"). This is the simplest task and was the focus of ImageNet.
2. **Classification + Localization**: Assign a label *and* predict a bounding box around the object. The model must answer both *what* and *where*.
3. **Object Detection**: Identify and localize *multiple* objects in a single image, each with its own bounding box and label. Architectures like YOLO (You Only Look Once) and Faster R-CNN are used.
4. **Instance Segmentation**: Go beyond bounding boxes to produce **pixel-level masks** for each object instance. Mask R-CNN is the landmark architecture for this task. This enables applications like autonomous driving, where the system must know the exact shape of every pedestrian and vehicle.

1.7 Generative Adversarial Networks (GANs)

While the previous sections focused on **discriminative** models—models that learn to classify or predict—**generative** models learn to *create* new data. Generative Adversarial Networks (GANs), introduced by Ian Goodfellow et al. in 2014, are one of the most influential generative architectures.

1.7.1 The Core Idea

A GAN consists of two neural networks that are trained simultaneously in a competitive game:

- **Generator** (G): Takes random noise $\mathbf{z} \sim p_z(\mathbf{z})$ as input and produces a synthetic sample $G(\mathbf{z})$. The generator is like a **counterfeiter** trying to produce convincing fake currency.
- **Discriminator** (D): Takes a sample (either real from the training data or fake from G) and outputs a probability $D(\mathbf{x}) \in [0, 1]$ indicating whether the sample is real. The discriminator is like a **detective** trying to distinguish real bills from counterfeits.

Training proceeds as a **zero-sum game**: G tries to fool D, while D tries to catch G. Over time, G becomes so good at generating realistic data that D can no longer distinguish real from fake.

1.7.2 Mathematical Formulation

The GAN training objective is a **minimax game**:

$$\min_G \max_D \; V(D, G) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \tag{1.1}$$

This objective has two components:

- **Discriminator's goal** (maximize V): Maximize $\log D(\mathbf{x})$ for real data (predict high probability) and $\log(1 - D(G(\mathbf{z})))$ for fake data (predict low probability).
- **Generator's goal** (minimize V): Minimize $\log(1-D(G(\mathbf{z})))$, which means making $D(G(\mathbf{z}))$ close to 1 (fooling the discriminator into thinking the fake is real).

The training alternates between:

1. **Update** D: Fix G, sample a batch of real data and fake data, and update D to better distinguish real from fake.
2. **Update** G: Fix D, sample noise, generate fake data, and update G to better fool D.

At the Nash equilibrium, the generator produces samples indistinguishable from real data, and the discriminator outputs $D(\mathbf{x}) = 0.5$ for all inputs.

Warning:

GAN Training Instability GANs are notoriously difficult to train. Common failure modes include:

- **Mode Collapse**: The generator produces only a few types of outputs, ignoring the diversity of the real data distribution.
- **Training Divergence**: The discriminator becomes too strong too quickly, providing no useful gradient signal to the generator.
- **Oscillation**: Neither network converges; they cycle without improving.

Techniques like Wasserstein loss (WGAN), spectral normalization, and progressive growing have been developed to stabilize training, but GANs remain significantly harder to train than standard supervised models.

1.7.3 Applications

GANs have enabled a wide range of creative and practical applications:

- **Image Generation**: Creating photorealistic faces of people who do not exist (StyleGAN).
- **DeepFakes**: Generating realistic video of people saying or doing things they never did—a powerful but ethically fraught application.
- **Data Augmentation**: Generating synthetic training data for domains where real data is scarce (medical imaging, rare defect detection).
- **Super-Resolution**: Upscaling low-resolution images to high-resolution (SRGAN).
- **Style Transfer**: Converting images from one artistic style to another (CycleGAN).
- **Text-to-Image Synthesis**: Generating images from textual descriptions, a precursor to modern diffusion-based models like DALL-E and Stable Diffusion.

1.8 The Rise of Natural Language Processing

Natural Language Processing (NLP) is the subfield of AI concerned with enabling machines to understand, generate, and interact using human language. While computer vision matured quickly with CNNs, NLP took longer to benefit from deep learning—but when the breakthrough came, it was revolutionary.

1.8.1 NLP Evolution Timeline

The evolution of NLP can be traced through a series of landmark developments:

- **2003**: Bengio et al. introduce **neural language models**—the first use of neural networks to model word sequences by learning dense vector representations.
- **2008**: Collobert and Weston demonstrate **multi-task learning** for NLP, showing that a single neural network can learn representations useful for multiple tasks simultaneously.
- **2013**: Mikolov et al. introduce **Word2Vec**, which efficiently learns dense word embeddings from large corpora. The famous analogy $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ captures the public imagination.
- **2014**: Sutskever et al. introduce **sequence-to-sequence** (Seq2Seq) learning with encoder-decoder RNNs, enabling machine translation.
- **2015**: Bahdanau et al. introduce the **attention mechanism**, allowing models to focus on relevant parts of the input rather than compressing everything into a fixed-length vector.
- **2017**: Vaswani et al. publish **"Attention Is All You Need,"** introducing the **Transformer** architecture—eliminating recurrence entirely and relying solely on attention mechanisms.
- **2018**: Pre-trained models emerge—**BERT** (Google) and **GPT** (OpenAI)—demonstrating that large-scale pre-training on unlabeled text, followed by fine-tuning, achieves state-of-the-art results on virtually every NLP benchmark.

1.8.2 Core NLP Tasks

NLP encompasses a diverse set of tasks:

- **Text Classification**: Assigning a label to a piece of text (spam detection, sentiment analysis, topic classification).
- **Information Extraction**: Identifying structured information from text, such as **Named Entity Recognition** (NER)—detecting persons, organizations, locations, dates.
- **Machine Translation**: Converting text from one language to another (e.g., English to Arabic).

- **Question Answering**: Given a question and a context passage, extracting or generating the answer.
- **Text Generation**: Producing coherent, contextually appropriate text—the core capability of large language models.
- **Conversational Agents**: Building systems that can engage in multi-turn dialogue (chatbots, virtual assistants).

1.8.3 Sequential Data and Models

Language is inherently **sequential**: the meaning of a word depends on the words that precede and follow it. This sequential nature extends to other data types as well:

- **Text**: Sequences of words or tokens
- **Audio**: Sequences of sound samples
- **Time Series**: Sequences of measurements over time (stock prices, sensor data)
- **DNA**: Sequences of nucleotides (A, C, G, T)
- **Video**: Sequences of image frames

Different tasks require different **input-output architectures**. The five fundamental sequence modeling patterns are:

1. **One-to-One**: Single input → single output. Standard classification (e.g., image → label). This is not truly sequential but is included for completeness.
2. **One-to-Many**: Single input → sequence output. Example: **image captioning**—a single image produces a sequence of words describing it.
3. **Many-to-One**: Sequence input → single output. Example: **sentiment analysis**—a sequence of words produces a single label (positive/negative).
4. **Many-to-Many (Aligned)**: Input and output sequences have the same length. Example: **video frame labeling**—each frame gets a label.
5. **Many-to-Many (Unaligned)**: Input and output sequences have different lengths. Example: **machine translation**—an English sentence of 5 words may translate to an Arabic sentence of 7 words. This requires an **encoder-decoder** architecture.

1.9 The Transformer Revolution (Preview)

For decades, the dominant paradigm for modeling sequential data was the **Recurrent Neural Network (RNN)** and its variants—the Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU). These models process sequences one token at a time, maintaining a hidden state $\mathbf{h}_t$ that accumulates information from previous tokens:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta)$$

While elegant in theory, RNNs suffer from three critical limitations:

1. **Sequential Processing**: Tokens must be processed one at a time, left to right. This prevents parallelization and makes training slow.
2. **Vanishing Gradients**: Over long sequences, gradients shrink exponentially during back-propagation, making it difficult to learn long-range dependencies. LSTMs and GRUs mitigate but do not eliminate this problem.
3. **Limited Memory**: Despite gating mechanisms, RNNs struggle to remember information from hundreds or thousands of tokens ago. The hidden state is a fixed-size bottleneck.

In 2017, Vaswani et al. published **"Attention Is All You Need"**, a paper that introduced the **Transformer** architecture and fundamentally changed the field. The key innovation was replacing recurrence with **self-attention**.

Self-attention allows every token in a sequence to attend to every other token **simultaneously**. Instead of processing sequentially ($O(N)$ serial steps), the entire sequence is processed in a single parallel computation. For a sequence of N tokens, self-attention computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

where $\mathbf{Q}$ (queries), $\mathbf{K}$ (keys), and $\mathbf{V}$ (values) are linear projections of the input, and d_k is the dimensionality of the keys. This formula is the heart of every modern language model, and we will derive it in full detail in Chapter 5.

The Transformer uses an **encoder-decoder** architecture:

- The **encoder** processes the input sequence and produces contextualized representations.
- The **decoder** generates the output sequence one token at a time, attending to both previously generated tokens and the encoder's representations.

The shift from sequential to parallel processing was a **paradigm shift**: training that once took weeks on RNNs could now be completed in hours on Transformers. This efficiency, combined with the model's ability to capture long-range dependencies via attention, made it possible to scale to billions of parameters—giving rise to GPT, BERT, and the entire ecosystem of large language models.

Key Insight:

From Sequential to Parallel: The Key Paradigm Shift The most important conceptual shift introduced by the Transformer is **abandoning sequential processing**. RNNs read a sentence like a human reads a book—one word at a time, left to right. Transformers read the entire page at once, allowing every word to directly interact with every other word. This is why Transformers can be trained on thousands of GPUs in parallel, while RNNs are fundamentally limited by their serial nature. Parallelism is not just a speedup—it is what makes billion-parameter models feasible.

1.10 Book Roadmap

This book takes a bottom-up approach to building Transformers from scratch. Each chapter introduces one essential component, building on the previous chapters to construct a complete understanding of modern language models.

- **Chapter 2: Tokenization.** How models "read" text. We explore how raw strings are broken into subword tokens using algorithms like Byte Pair Encoding (BPE) and SentencePiece, and why tokenization quality directly impacts model performance.
- **Chapter 3: Embeddings.** How models represent meaning as vectors. We study how tokens are mapped to dense numerical vectors that capture semantic relationships, and how positional embeddings encode word order.
- **Chapter 4: Recurrent Neural Networks (RNN, LSTM, GRU).** The predecessors to Transformers. We build sequential models that maintain context through hidden states, understand their limitations, and motivate the need for attention mechanisms.
- **Chapter 5: Self-Attention.** The core mechanism of the Transformer. We derive the scaled dot-product attention formula from first principles, implement multi-head attention, and show how it enables parallel processing of entire sequences.

- **Chapter 6: The Transformer.** Putting it all together. We assemble encoder and decoder blocks from attention, feed-forward networks, layer normalization, and residual connections to build a complete Transformer from scratch.

By the end of this book, you will understand not just *how* to use Transformers, but *why* they work—from the mathematical foundations to the implementation details.

1.11 Conclusion

This chapter has traced the journey from the broad ambitions of artificial intelligence to the specific breakthroughs of deep learning. We have seen how machine learning flipped the traditional programming paradigm—replacing hand-coded rules with learned patterns—and how deep learning extended this revolution to unstructured data like images and text.

We examined the three learning paradigms (supervised, unsupervised, and reinforcement learning), compared traditional ML with deep learning across five key dimensions, and explored how CNNs conquered computer vision through the ImageNet challenge. We introduced GANs as a powerful generative framework and traced the evolution of NLP from rule-based systems to the Transformer architecture.

The Transformer, with its self-attention mechanism and parallel processing capability, represents the most significant architectural innovation in modern AI. It is the foundation of GPT, BERT, LLaMA, and virtually every state-of-the-art language model in use today.

In the next chapter, we begin our bottom-up construction of Transformers by studying **tokenization**—the critical first step in teaching a machine to read.

2

Tokenization for Language Modeling

2.1 Introduction

Imagine trying to teach a computer to speak English. You can't just feed it raw audio waveforms or entire paragraphs at once; it's too complex. Instead, you break the language down into manageable **units of meaning**—words, syllables, or sounds.

Tokenization is the process of splitting text into these meaningful units, called **tokens**, and assigning each unit a unique linguistic identity. It is crucial to distinguish between the two key steps in this process:

1. **Tokenization (Segmentation)**: The linguistic act of breaking a string into pieces (tokens). Ideally, these pieces capture the **morphological structure** of the language.
   - Example: The word "unhappiness" is split into ‘["un", "happi", "ness"]‘.
   - This captures the prefix ‘un-‘ (negation), the root ‘happi‘ (emotion), and the suffix ‘-ness‘ (state of being). The model learns these components individually, allowing it to understand new words like "unfriendly" or "sadness" by recombining known parts.
2. **Numerical Encoding (Indexing)**: The computational act of mapping each unique token to a specific integer ID from a fixed vocabulary.
   - Example: ‘["un", "happi", "ness"]‘ → ‘[512, 3042, 109]‘.
   - The model takes these integers as input, not the text itself.

Before we dive into the complex architectures of Transformers, we must master how they "read" text. This chapter explores how we move from raw characters to intelligent subword tokens that power models like GPT-4.

2.1.1 The LLM Pipeline

Large Language Models (LLMs) like GPT-4 and Llama do not understand text directly. They are mathematical functions that operate on numerical vectors. To bridge the gap between human language and machine representation, we use a process called **tokenization**.

The pipeline for an LLM is as follows (see Figure 2.1):

1. **Input Text**: Raw natural language (e.g., "The mouse like eating").
2. **Tokenization**: The text is broken into smaller units called **tokens** (e.g., ["The", "mouse", "like", "eating"]).
3. **Token Embeddings**: These tokens are mapped to dense numerical vectors (embeddings) combined with positional information.
4. **Transformer Block**: The core neural network processes these embeddings to understand relationships (e.g., relating "mouse" to "eating").
5. **Next Token Prediction**: The model predicts the probability of the next token (e.g., "cheese": 85%).

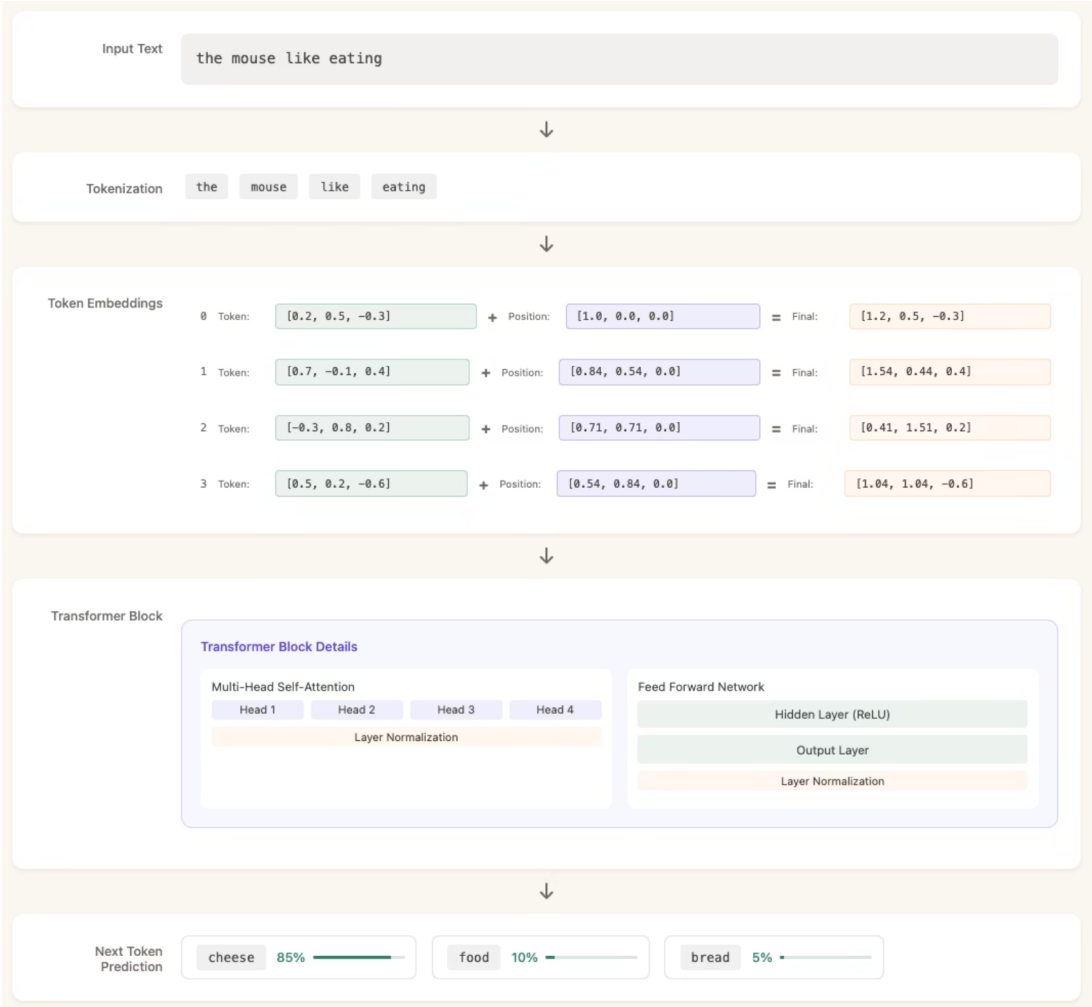

Figure 2.1: The Language Modeling Pipeline

2.2 Mathematical Formalism

Formally, we can define a tokenizer as a function Φ that maps a string sequence $\mathcal{S}$ from a character set Σ (e.g., Unicode) to a sequence of integers $\mathcal{T} \in \mathbb{Z}^N$.

$$\Phi : \Sigma^* \to \{0, 1, \ldots, |\mathcal{V}| - 1\}^N$$

Where:

- Σ^* represents the set of all possible strings.
- $\mathcal{V}$ is the **Vocabulary**, a finite set of discrete tokens.
- N is the length of the tokenized sequence.

The goal of an optimal tokenizer is often to minimize the sequence length N while keeping the vocabulary size $|\mathcal{V}|$ within computationally feasible limits (e.g., $< 100,000$).

Key Insight:

The "Goldilocks" Trade-off Tokenization is a balancing act.

- **Too Granular (Characters)**: Vocabulary is small ($\sim$100), but sequences are huge (N is large). This makes training slow and context limited.

- **Too Coarse (Words)**: Sequences are short (N is small), but vocabulary is massive ($\sim$1M+). This leads to sparse embeddings and OOV issues.
- **Just Right (Subwords)**: Algorithms like BPE find the sweet spot—keeping common words whole while breaking rare ones.

2.3 From Vocabulary to Indices

Tokenization relies on a pre-defined **Vocabulary** ($\mathcal{V}$), which is a list of all unique tokens the model knows.

2.3.1 The Lookup Process

1. **Vocabulary**: A set of tokens like ‘["hello", "world", "!"]‘.
2. **Indexing**: Each token is assigned a unique integer ID.
   - "hello" $\rightarrow$ 0
   - "world" $\rightarrow$ 1
   - "!" $\rightarrow$ 2
3. **Encoding**: The input sequence is converted to these IDs.

$$\text{"hello world!"} \rightarrow [0, 1, 2]$$

2.3.2 Handling Unknown Words (OOV)

A major challenge is handling words that are not in the vocabulary (**Out-Of-Vocabulary** or OOV).

- **Naïve Approach**: Replace with a special ‘[UNK]‘ token. This destroys information.
- **Subword Approach**: Break the unknown word into smaller known pieces.

$$\text{"universe"} \rightarrow [\text{"uni"}, \text{"verse"}]$$

  This ensures the model can process any text, provided the subwords are in the vocabulary.

2.4 Why Tokenization Matters

"The better the tokenization, the smarter the model." Poor tokenization leads to:

- **Spelling Errors**: The model fails to see the internal structure of words.
- **Arithmetic Mistakes**: If numbers are split arbitrarily (e.g., "123" $\rightarrow$ ["1", "23"]), the model struggles to do math.
- **Multilingual Inefficiency**: Languages like Korean or Arabic might be fragmented into excessive numbers of tokens, diluting semantic meaning.

Key Insight:

Why "123" should be "1", "2", "3" or "123"? For a model to learn math, consistent tokenization is key. If "100" is one token but "101" is two tokens ("10", "1"), the model struggles to generalize place value. Newer tokenizers (like GPT-4’s) try to group digits more consistently (e.g., keeping up to 3 digits together) to improve arithmetic performance.

2.5 Evolution of GPT Tokenizers: A Comparative Analysis

The evolution from GPT-2 to GPT-4 demonstrates a clear trend towards efficiency, standardization, and better handling of code and multilingual text. We analyze this progression through three generations of tokenizers.

2.5.1 GPT-2 Tokenizer (50k Vocabulary)

The GPT-2 tokenizer, based on byte-level BPE, was a pioneering approach that allowed models to process any text string without needing a massive vocabulary. It utilized a vocabulary size of approximately 50,257 tokens.

However, it had significant inefficiencies. As seen in Figure 2.2, the tokenizer often fails to keep common words or numbers as single units.

1. **Inefficient Splitting**: Common words, especially in non-English languages, were often fragmented into multiple small tokens.
2. **Whitespace Handling**: It treated spaces as part of the following word in some contexts but not others, leading to inconsistent tokenization of similar phrases.
3. **High Token Count**: In the visual example, the text requires **232 tokens**. This high count limits the amount of information that can fit into the model's context window.

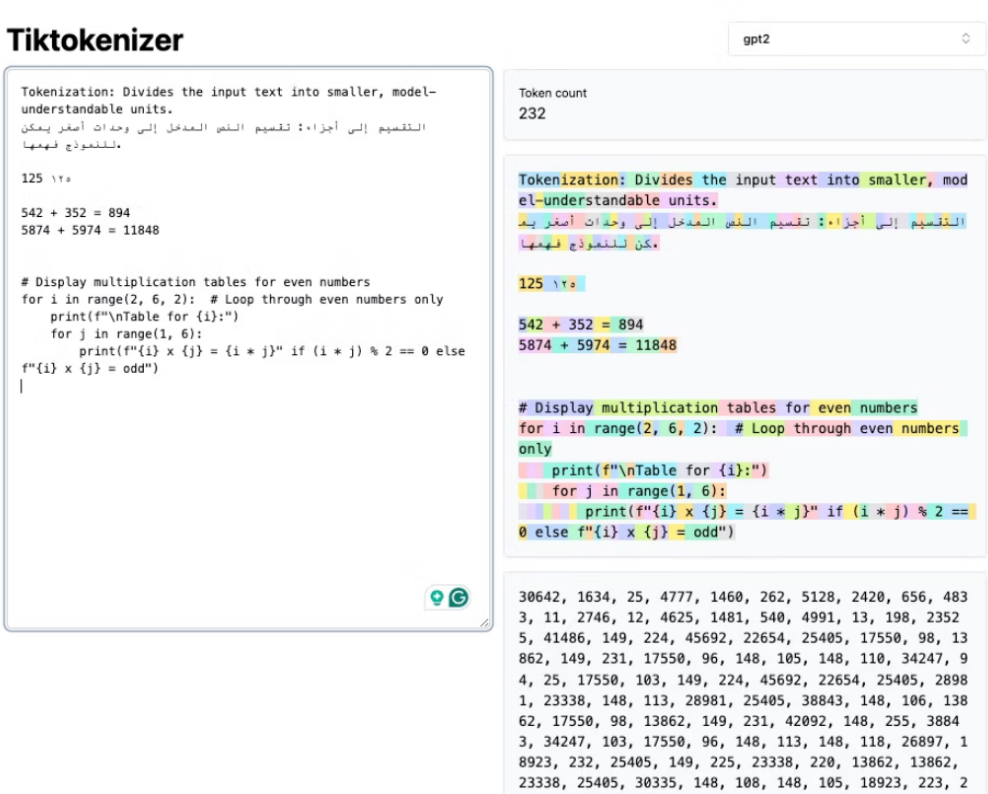

Figure 2.2: GPT-2 Tokenization: Note the inefficient splitting and high token count (232).

2.5.2 GPT-3.5 (Turbo) Tokenizer

GPT-3.5 (used in the original ChatGPT) represented an intermediate step. While it improved upon the raw efficiency of GPT-2, it still shared many underlying characteristics.

- **Vocabulary**: It largely maintained the vocabulary structure of the GPT-3 era.

- **Performance**: It achieved better compression than GPT-2 but still struggled with complex code structures and certain multilingual scripts compared to the latest generation.

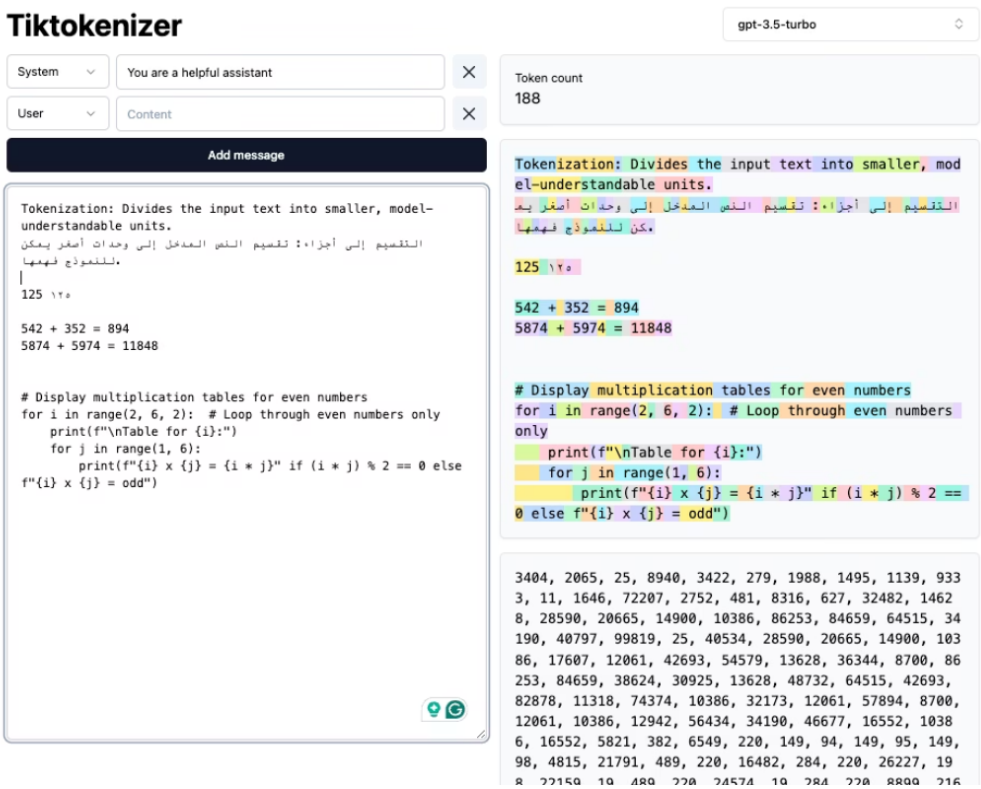

Figure 2.3: GPT-3.5 Tokenization: An improvement over GPT-2.

2.5.3 GPT-4 Tokenizer (cl100k_base)

GPT-4 introduced the **cl100k_base** tokenizer, which represents a significant leap in efficiency and semantic alignment. The vocabulary size was effectively doubled to ∼100,000 tokens.

This expansion allowed for several critical optimizations (see Figure 2.4):

1. **Digit Grouping**: Unlike GPT-2, which split numbers arbitrarily (e.g., "123" → "1", "23"), GPT-4 groups up to three digits into a single token. This consistency dramatically aids the model's ability to perform arithmetic.
2. **Code Efficiency**: The tokenizer includes specific tokens for common whitespace patterns (indentation) and programming keywords, reducing the token count for code by 30-40%.
3. **Multilingual Support**: The larger vocabulary allocates more space to common tokens in languages beyond English, making the model more "native" to global languages.
4. **Semantic Density**: By compressing text into fewer tokens (shorter sequences), the model can attend to longer contexts with the same compute budget.

2.6 Unicode and Byte-Level Processing

2.6.1 What is Unicode?

Unicode is the standard for text encoding, assigning a unique **Code Point** to over 150,000 characters (e.g., 'A' is 65, 'U+1F60A' is 128522).

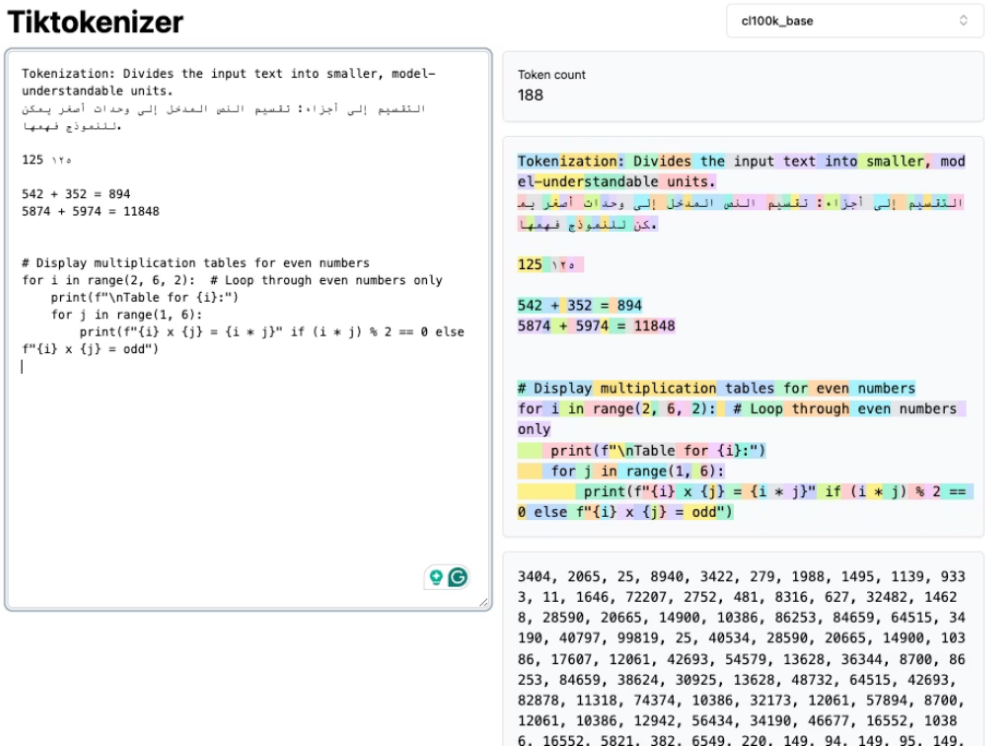

Figure 2.4: GPT-4 Tokenization: Highly compact and efficient (note the digit grouping).

Feature	GPT-2	GPT-4
Vocabulary Size	50,257	100,000
Efficiency	Lower (longer sequences)	Higher (shorter sequences)
Numbers	Splits arbitrarily	Groups digits effectively
Python Code	Spaces are separate tokens	Spaces merged efficiently

Table 2.1: Comparison of GPT-2 and GPT-4 Tokenizers

2.6.2 Why Not Use Unicode Code Points?

1. **Vocabulary Size**: 150,000+ tokens is too large for the model's output layer (embedding table becomes massive).
2. **Dynamic Standard**: New emojis and characters are added every year.
3. **No Semantics**: Code points don't capture the meaning of subwords like "un-" or "-ing".

2.6.3 Why Not Use Raw UTF-8 Bytes?

Using raw bytes (256 possibilities) solves the vocabulary size problem but introduces new issues:

1. **Sequence Length**: A single character like 'U+1F60A' becomes 4 bytes (`[240, 159, 152, 131]`). Text becomes 4-5x longer.
2. **Inefficient Attention**: Transformer complexity scales quadratically ($O(N^2)$). 4x length means 16x compute cost.

Key Insight:

The Solution We need a middle ground: **Subword Tokenization**. It uses bytes to ensure no OOV, but merges them to keep sequences short.

2.7 Tokenization Paradigms

2.7.1 1. Character-Level

- **Pros**: Tiny vocabulary (100 tokens), no OOV.
- **Cons**: Extremely long sequences, lacks semantic meaning per token.

2.7.2 2. Word-Level

- **Pros**: High semantic value, short sequences.
- **Cons**: Massive vocabulary, many OOV words, fails on rich morphology.

2.7.3 3. Subword-Level (The Standard)

Balances the two. It keeps frequent words whole ("the", "apple") and breaks rare words ("un-friendly" → "un", "friend", "ly").

Challenge: Arabic Morphology

Arabic is morphologically rich. A single word often contains a prefix, root, and suffix. For example, the word "Wakatabna" means "And we wrote". Traditional word tokenization fails here because every combination matches a unique dictionary word, leading to an explosion of vocabulary. Subword tokenization is essential for Arabic to split these components.

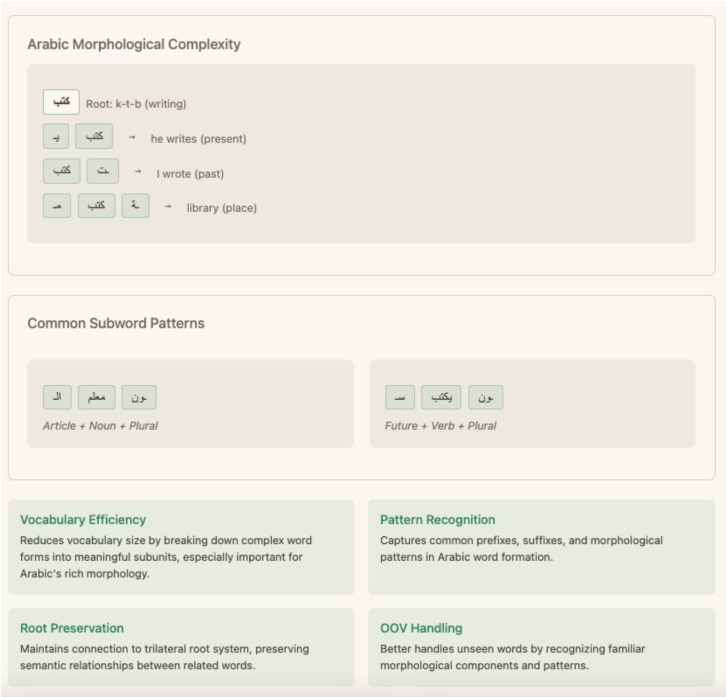

Figure 2.5: Arabic Morphology Example

2.8 Algorithm: Byte Pair Encoding (BPE)

BPE is a compression algorithm used by GPT-2, GPT-3, and GPT-4.

2.8.1 How It Works

1. **Initialize**: Start with a vocabulary of individual characters (or bytes).
2. **Count Pairs**: Find the most frequent adjacent pair of tokens in the corpus.
3. **Merge**: Combine them into a new single token. Add to vocabulary.
4. **Repeat**: Continue until the target vocabulary size is reached.

Example:

BPE Walkthrough **Corpus**: "low lower lowest"
Initial: [l, o, w, _, l, o, w, e, r, _, l, o, w, e, s, t]
Most Frequent: ('l', 'o')
Merge 1: Create token 'lo'. Corpus: [lo, w, _, lo, w, e, r, _, lo, w, e, s, t]
Most Frequent: ('lo', 'w')
Merge 2: Create token 'low'. Corpus: [low, _, low, e, r, _, low, e, s, t]

2.9 SentencePiece

SentencePiece (by Google) is used in LLaMA and Mistral.

- **Lossless**: Treats whitespace as a distinct symbol (e.g., U+2581 _).
- **Raw Input**: Does not require pre-tokenization (splitting by space) like original BPE.
- **Byte Fallback**: If a character is not in the vocabulary, it falls back to its UTF-8 byte representation, ensuring no [UNK] tokens.

2.10 Special Tokens

Special tokens add structure to the model's input.

- `<|endoftext|>`: Marks the end of a document.
- `<|im_start|>`, `<|im_end|>`: Delimit turns in a conversation (User vs. Assistant).
- `<|fim_prefix|>`: Used for code completion (Fill-In-The-Middle).

2.11 Tokenization Performance Metrics

2.11.1 Intrinsic Metrics

These measure the quality of the vocabulary itself.

1. Fertility

Average number of tokens per word. Lower is generally better (closer to 1.0 means words are kept whole).

$$\text{Fertility} = \frac{\text{Total Tokens}}{\text{Total Words}}$$

Figure 2.6: Fertility Rate Comparison

2. Single Word Token Performance

Percentage of words that are tokenized as a single unit. High values indicate a vocabulary well-suited to the domain.

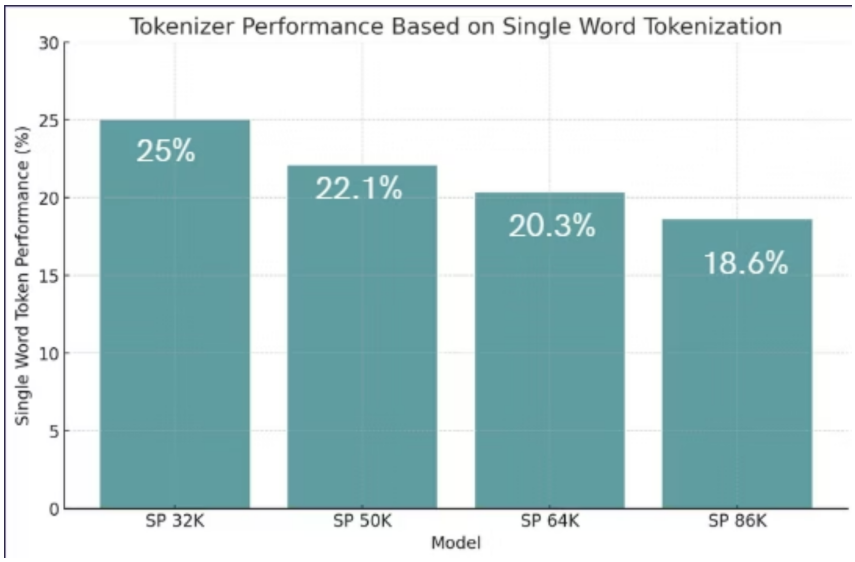

Figure 2.7: Single Word Token Performance

3. Root Token Performance

Ability to protect the root of the word. For "running", we want ["run", "ning"], not ["ru", "nning"]. Capturing the root preserves semantics.

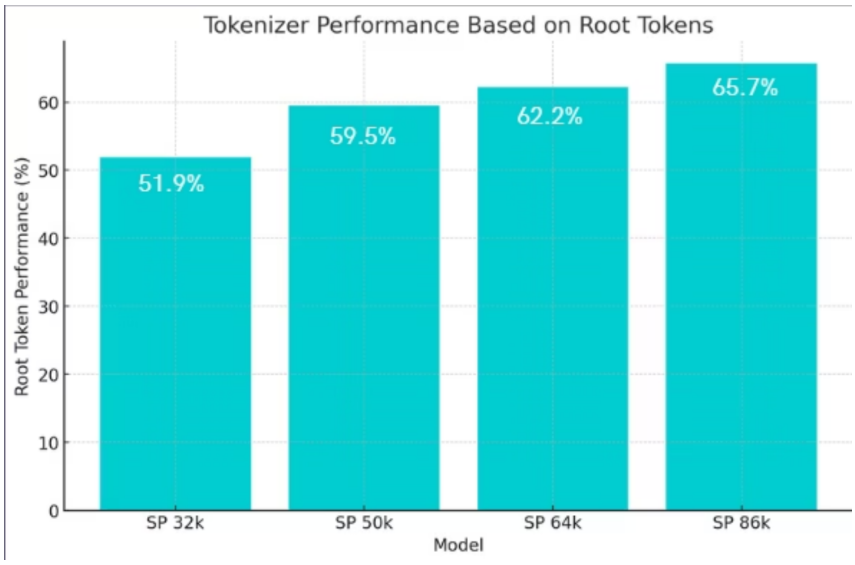

Figure 2.8: Root Token Performance

4. Suffix Precision

When a word is split, is the suffix a real linguistic suffix? (e.g., "-ing" is good, "-ng" is bad).

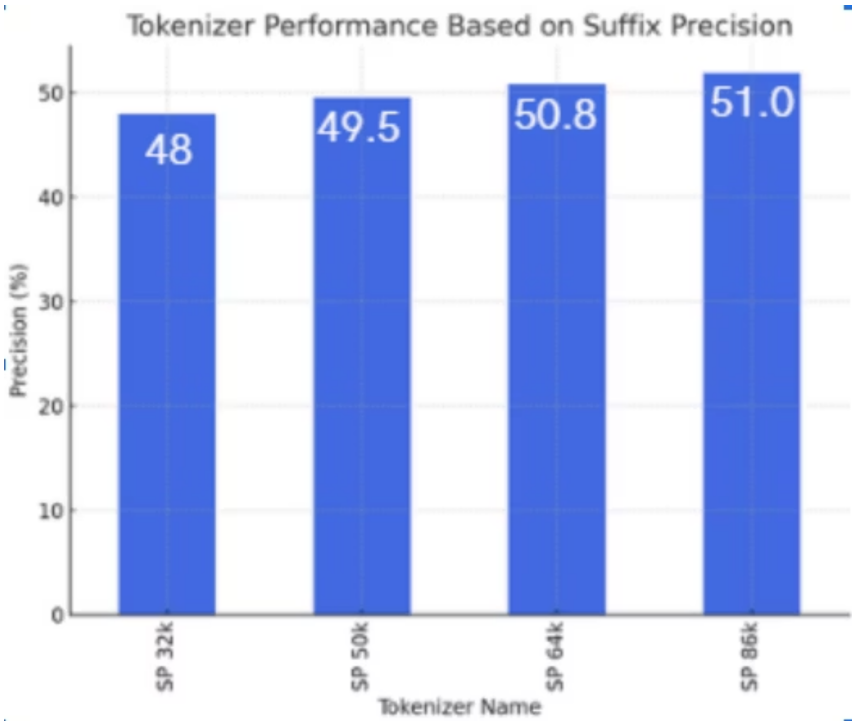

Figure 2.9: Suffix Precision

2.11.2 Extrinsic Metrics

- **Downstream Performance**: Accuracy on real tasks (translation, summarization).
- **Computational Cost**: Depends on sequence length (Fertility).

2.12 Conclusion

Tokenization is more than just string splitting; it is a critical compression step that defines how an LLM views the world. By moving from simple heuristics to data-driven algorithms like BPE and SentencePiece, we enable models to handle diverse languages, code, and complex morphology efficiently.

3

Word Embedding and Vector Representation

3.1 The Mathematical Foundation: Vectors

Before we can represent words as numbers, we must understand the mathematical object that makes this possible: the **vector**. Every modern AI system—whether it processes images, text, or audio—ultimately operates on vectors. This section builds the mathematical intuition that underpins all of NLP.

3.1.1 Why Everything in AI Must Be a Number

Neural networks are, at their core, compositions of matrix multiplications and nonlinear functions. They cannot process symbols, characters, or pixels directly. Every input must first be converted into a **numerical representation**—a vector of real numbers. This constraint is not a limitation; it is what gives AI its power. Once data is represented as vectors in a high-dimensional space, we can measure similarity, compute distances, and learn transformations using the tools of linear algebra and calculus.

Key Insight:

The Universal Language of AI Regardless of the domain—computer vision, natural language processing, speech recognition, or robotics—the internal representation is always the same: vectors of real numbers. Learning to think in terms of vectors is learning to think like a neural network.

3.1.2 Vectors in Two and Three Dimensions

Definition: Vector

A **vector** is an ordered list of numbers that defines a point or direction in space. A vector in $\mathbb{R}^n$ has n components:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n$$

Each component v_i represents the magnitude along the i-th axis of the space.

In two dimensions, a vector $\mathbf{v} = [v_1, v_2]$ can be visualized as an arrow from the origin $(0, 0)$ to the point (v_1, v_2) in the Cartesian plane. For example, $\mathbf{v} = [3, 4]$ points 3 units along the x-axis and 4 units along the y-axis.

In three dimensions, a vector $\mathbf{v} = [v_1, v_2, v_3]$ extends this to include a z-axis. For instance, $\mathbf{v} = [1, 2, 3]$ defines a direction in 3D space.

While we can visualize 2D and 3D vectors geometrically, the mathematics generalizes seamlessly to any number of dimensions. In NLP, word embeddings typically live in spaces of 100 to 1000 dimensions, far beyond visual intuition but governed by the same algebraic rules.

Example:

A 2D Vector Consider the vector $\mathbf{v} = [3, 4]$.

- It has 2 components: $v_1 = 3$ and $v_2 = 4$.
- Geometrically, it is an arrow from the origin to the point $(3, 4)$.
- Its length (magnitude) is $\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$.

3.1.3 Feature Vectors: Ordered Lists of Numerical Attributes

In machine learning, a **feature vector** is a vector where each component represents a measurable attribute (feature) of a data point. The key requirement is that the ordering is consistent: the first component always represents the same feature across all data points.

Definition: Feature Vector

A **feature vector** is an ordered list of numerical attributes $\mathbf{x} = [x_1, x_2, \ldots, x_n]$ where each x_i measures a specific, pre-defined property of the data point. Two data points can be compared by computing operations on their feature vectors.

Example:

Feature Vector for a House Suppose we represent houses using four features: area (sq ft), number of bedrooms, age (years), and distance to city center (km).

$$\mathbf{h}_1 = [1500, 3, 10, 5.2], \quad \mathbf{h}_2 = [2200, 4, 2, 3.1]$$

Each position encodes the same feature across both houses. We can now compute how "similar" these houses are using vector operations—something impossible with raw text descriptions like "a cozy 3-bedroom home."

3.1.4 Vectors in Computer Vision

In computer vision, images are naturally numerical. A grayscale image is a 2D matrix of pixel intensities, where each value lies in the range $[0, 255]$ (0 = black, 255 = white). A color image extends this with three channels—Red, Green, and Blue (RGB)—resulting in a 3D tensor of shape (Height $\times$ Width $\times$ 3).

To feed an image into a neural network, we **flatten** this tensor into a single long vector. For a 28×28 grayscale image (like MNIST digits), the feature vector has $28 \times 28 = 784$ dimensions:

$$\mathbf{x}_{\text{image}} = [x_1, x_2, \ldots, x_{784}] \in [0, 255]^{784}$$

For a 224×224 RGB image (like those in ImageNet), the vector has $224 \times 224 \times 3 = 150{,}528$ dimensions.

Key Insight:

Images Are "Easy" to Vectorize Images come with a natural numerical representation: pixel values. The challenge in computer vision is not *how* to represent images as vectors, but how to extract *meaningful* features from those vectors. Text, by contrast, has no inherent numerical form—which is why word embedding is such a fundamental problem in NLP.

3.1.5 Vectors in Natural Language Processing

Unlike images, words are discrete symbols with no inherent numerical meaning. The letter "c" in "cat" does not carry a numeric value the way a pixel does. The central challenge of NLP is therefore: *How do we convert words into vectors that capture their meaning?*

This is the problem of **word embedding**—mapping each word to a dense vector in $\mathbb{R}^d$ such that semantically similar words are mapped to nearby points:

$$\text{“cat”} \to [0.25, -0.13, 0.87, \dots] \in \mathbb{R}^{300}$$

$$\text{“kitten”} \to [0.27, -0.11, 0.85, \dots] \in \mathbb{R}^{300}$$

The rest of this chapter is devoted to understanding how these vectors are constructed and what mathematical operations make them useful.

3.2 Vector Operations

Now that we understand what vectors are, we need the algebraic operations that allow us to manipulate them. These operations are not mere abstractions—they form the computational backbone of every neural network.

3.2.1 Addition and Subtraction

Definition: Vector Addition and Subtraction

Given two vectors $\mathbf{v}, \mathbf{w} \in \mathbb{R}^n$, their sum and difference are computed component-wise:

$$\mathbf{v} + \mathbf{w} = \begin{bmatrix} v_1 + w_1 \\ v_2 + w_2 \\ \vdots \\ v_n + w_n \end{bmatrix}, \qquad \mathbf{v} - \mathbf{w} = \begin{bmatrix} v_1 - w_1 \\ v_2 - w_2 \\ \vdots \\ v_n - w_n \end{bmatrix}$$

Example:

Vector Addition and Subtraction Let $\mathbf{v} = [2, 4, 6]$ and $\mathbf{w} = [1, 0, 1]$.
Addition:

$$\mathbf{v} + \mathbf{w} = [2 + 1,\ 4 + 0,\ 6 + 1] = [3, 4, 7]$$

Subtraction:

$$\mathbf{v} - \mathbf{w} = [2 - 1,\ 4 - 0,\ 6 - 1] = [1, 4, 5]$$

Geometric Intuition. Vector addition corresponds to placing two arrows tip-to-tail: the result is an arrow from the start of the first to the end of the second (the parallelogram rule). Vector subtraction $\mathbf{v} - \mathbf{w}$ produces a vector that points from $\mathbf{w}$ to $\mathbf{v}$—it encodes the *difference* or

offset between the two points. This geometric view is precisely what makes Word2Vec analogies work: the vector $\vec{\text{King}} - \vec{\text{Man}}$ captures the "royalty" direction, independent of gender.

3.2.2 Scalar Multiplication

Definition: Scalar Multiplication

Given a scalar $c \in \mathbb{R}$ and a vector $\mathbf{v} \in \mathbb{R}^n$, the scalar product is:

$$c \cdot \mathbf{v} = \begin{bmatrix} c \cdot v_1 \\ c \cdot v_2 \\ \vdots \\ c \cdot v_n \end{bmatrix}$$

Example:

Scalar Multiplication Let $c = 3$ and $\mathbf{v} = [2, 4, 6]$.

$$c \cdot \mathbf{v} = 3 \cdot [2, 4, 6] = [3 \times 2,\ 3 \times 4,\ 3 \times 6] = [6, 12, 18]$$

Geometric Intuition. Scalar multiplication scales the vector's magnitude without changing its direction (if $c > 0$) or reverses its direction (if $c < 0$). When $c = 0$, the result is the zero vector. When $|c| = 1$, the magnitude is preserved but the direction may flip. This operation is fundamental to normalization (Section 3.4), where we divide a vector by its magnitude to obtain a unit vector.

3.3 The Dot Product

The dot product is arguably the single most important operation in deep learning. It appears in attention mechanisms, loss functions, similarity computations, and every layer of every neural network.

Definition: Dot Product

The **dot product** (or **inner product**) of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ is a scalar defined as:

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} x_i\, y_i = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$$

In matrix notation, this can be written compactly as:

$$\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y}$$

where $\mathbf{x}^\top$ is the transpose of $\mathbf{x}$ (converting a column vector to a row vector), and the operation becomes a standard matrix multiplication yielding a 1×1 scalar.

Example:

Computing the Dot Product Let $\mathbf{x} = [2, 3, 4]$ and $\mathbf{y} = [1, 0, -1]$.

$$\begin{aligned} \mathbf{x} \cdot \mathbf{y} &= (2)(1) + (3)(0) + (4)(-1) \\ &= 2 + 0 + (-4) \\ &= -2 \end{aligned}$$

The negative result tells us that these two vectors point in somewhat opposing directions.

3.3.1 Properties of the Dot Product

The dot product satisfies several important algebraic properties. For all vectors $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{R}^n$ and scalar $c \in \mathbb{R}$:

1. **Commutative**: $\mathbf{x} \cdot \mathbf{y} = \mathbf{y} \cdot \mathbf{x}$
2. **Distributive**: $\mathbf{x} \cdot (\mathbf{y} + \mathbf{z}) = \mathbf{x} \cdot \mathbf{y} + \mathbf{x} \cdot \mathbf{z}$
3. **Scalar Compatibility**: $(c\,\mathbf{x}) \cdot \mathbf{y} = c\,(\mathbf{x} \cdot \mathbf{y})$
4. **Self-dot product**: $\mathbf{x} \cdot \mathbf{x} = \|\mathbf{x}\|^2 \geq 0$, with equality if and only if $\mathbf{x} = \mathbf{0}$

3.3.2 Geometric Meaning: Alignment Between Vectors

The dot product has a profound geometric interpretation. It measures how much two vectors *align* with each other. The relationship is captured by:

$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\|\ \|\mathbf{y}\|\ \cos\theta$$

where θ is the angle between the two vectors. This formula reveals three critical cases:

1. **Equal (parallel) vectors** ($\theta = 0°$): The dot product equals $\|\mathbf{v}\|^2$, the maximum possible value for vectors of that magnitude. The vectors point in the same direction.

$$\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|^2 \quad \text{(maximum alignment)}$$

2. **Perpendicular (orthogonal) vectors** ($\theta = 90°$): The dot product equals **zero**. The vectors share no component in common.

$$\mathbf{x} \cdot \mathbf{y} = 0 \quad \text{(no alignment)}$$

3. **Opposite (anti-parallel) vectors** ($\theta = 180°$): The dot product equals $-\|\mathbf{v}\|^2$, the minimum possible value. The vectors point in opposite directions.

$$\mathbf{v} \cdot (-\mathbf{v}) = -\|\mathbf{v}\|^2 \quad \text{(maximum opposition)}$$

Key Insight:

Why the Dot Product Matters in NLP When we represent words as vectors, the dot product tells us how "aligned" two words are. A large positive dot product means the words are used in similar contexts (semantically related). A dot product near zero means the words are unrelated. A negative dot product means the words appear in contrasting contexts. This is the mathematical foundation of semantic similarity, and it is the core operation inside the attention mechanism of Transformers.

Example:

Dot Product of Perpendicular Vectors Let $\mathbf{a} = [1, 0]$ and $\mathbf{b} = [0, 1]$.

$$\mathbf{a} \cdot \mathbf{b} = (1)(0) + (0)(1) = 0$$

These vectors are perpendicular—they share no directional component. This is exactly what happens with one-hot encoded words: every pair of distinct words has a dot product of zero, meaning the representation encodes *no* similarity information whatsoever.

3.4 Vector Normalization

When comparing vectors, we often care about their *direction* (what they represent) rather than their *magnitude* (how large they are). Normalization rescales a vector to have unit length, isolating its directional information.

3.4.1 Why Normalize?

Consider two word vectors: $\mathbf{v}_{\text{cat}} = [0.5, 0.3, 0.8]$ and $\mathbf{u}_{\text{kitten}} = [5.0, 3.0, 8.0]$. These vectors point in the *same* direction (one is simply 10 times the other), yet their dot product $\mathbf{v} \cdot \mathbf{u} = 2.5 + 0.9 + 6.4 = 9.8$ is large primarily because of the magnitude difference, not because of any special semantic alignment. Normalization eliminates this confound.

3.4.2 The L2 Norm (Euclidean Norm)

Definition: L2 Norm

The **L2 norm** (Euclidean norm) of a vector $\mathbf{x} \in \mathbb{R}^n$ is the square root of the sum of the squares of its components:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

It measures the "straight-line" distance from the origin to the point $\mathbf{x}$.

Definition: L2 Normalization

A vector $\mathbf{x}$ is **normalized** (converted to a unit vector) by dividing it by its L2 norm:

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}$$

The resulting vector $\hat{\mathbf{x}}$ satisfies $\|\hat{\mathbf{x}}\|_2 = 1$.

Example:

L2 Normalization Step by Step Let $\mathbf{v} = [3, 4]$.

Step 1: Compute the L2 norm.

$$\|\mathbf{v}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Step 2: Divide each component by the norm.

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{5} = \left[\frac{3}{5}, \frac{4}{5}\right] = [0.6,\ 0.8]$$

Step 3: Verify.

$$\|\hat{\mathbf{v}}\|_2 = \sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = \sqrt{1.0} = 1 \checkmark$$

The normalized vector $[0.6, 0.8]$ has the same direction as $[3, 4]$ but unit length.

3.4.3 The L1 Norm (Manhattan Norm)

Definition: L1 Norm

The **L1 norm** (Manhattan norm or taxicab norm) of a vector $\mathbf{x} \in \mathbb{R}^n$ is the sum of the absolute values of its components:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i| = |x_1| + |x_2| + \cdots + |x_n|$$

It measures the distance traveled along axis-aligned paths (like navigating a city grid).

Example:

L1 Norm Computation Let $\mathbf{v} = [3, -4, 2]$.

$$\|\mathbf{v}\|_1 = |3| + |-4| + |2| = 3 + 4 + 2 = 9$$

Compare with the L2 norm: $\|\mathbf{v}\|_2 = \sqrt{9 + 16 + 4} = \sqrt{29} \approx 5.39$. The L1 norm is always greater than or equal to the L2 norm.

3.4.4 Comparison of L1 and L2 Norms

Property	L1 Norm	L2 Norm
Formula	$\sum \lvert x_i \rvert$	$\sqrt{\sum x_i^2}$
Geometric Meaning	Grid distance	Straight-line distance
Sensitivity to Outliers	Less sensitive	More sensitive
Sparsity	Promotes sparse solutions	Does not promote sparsity
Common Use	Feature selection, LASSO	Distance metrics, normalization

Table 3.1: Comparison of L1 and L2 Norms

3.4.5 Implementation in NumPy

Computing norms and normalization in Python is straightforward with NumPy:

```python
import numpy as np

v = np.array([3, 4])

# L2 Norm
l2_norm = np.linalg.norm(v)          # 5.0
unit_v = v / l2_norm                  # [0.6, 0.8]

# L1 Norm
l1_norm = np.linalg.norm(v, ord=1)   # 7.0

# Verify unit length
print(np.linalg.norm(unit_v))        # 1.0
```

3.5 Cosine Similarity

While the dot product measures alignment, it is confounded by vector magnitudes. **Cosine similarity** removes this confound by normalizing for length, providing a pure measure of directional similarity.

Definition: Cosine Similarity

The **cosine similarity** between two non-zero vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ is defined as:

$$\text{cos_sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2} = \frac{\sum_{i=1}^n x_i \, y_i}{\sqrt{\sum_{i=1}^n x_i^2} \, \sqrt{\sum_{i=1}^n y_i^2}}$$

The cosine similarity ranges from -1 to $+1$:

Value	Interpretation
$+1$	Vectors point in exactly the same direction (identical orientation)
0	Vectors are perpendicular (completely unrelated)
-1	Vectors point in exactly opposite directions (maximally dissimilar)

Table 3.2: Interpreting Cosine Similarity Values

Example:

Cosine Similarity: Step-by-Step Computation Let $\mathbf{a} = [3, 4]$ and $\mathbf{b} = [4, 3]$.

Step 1: Compute the dot product.

$$\mathbf{a} \cdot \mathbf{b} = (3)(4) + (4)(3) = 12 + 12 = 24$$

Step 2: Compute the magnitudes.

$$\|\mathbf{a}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5, \qquad \|\mathbf{b}\|_2 = \sqrt{4^2 + 3^2} = \sqrt{25} = 5$$

Step 3: Compute cosine similarity.

$$\text{cos_sim}(\mathbf{a}, \mathbf{b}) = \frac{24}{5 \times 5} = \frac{24}{25} = 0.96$$

The value 0.96 (very close to 1) indicates that the vectors $[3, 4]$ and $[4, 3]$ point in nearly the same direction, despite having different component values.

3.5.1 Why Cosine Similarity Over the Dot Product for NLP

The raw dot product is magnitude-dependent: $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\|\|\mathbf{y}\| \cos\theta$. If one word appears 1000 times and another appears 10 times, their embedding magnitudes may differ dramatically, inflating or deflating the dot product regardless of semantic relationship.

Cosine similarity eliminates this problem by dividing out the magnitudes, leaving only $\cos\theta$—the angle between the vectors. Two words that are used in identical contexts will have cosine similarity ≈ 1 regardless of their frequency in the corpus.

Warning:

When Cosine Similarity Fails Cosine similarity measures only the *angle* between vectors. Two vectors can have cosine similarity of 1 (pointing in the same direction) but vastly different magnitudes. In some applications—such as when magnitude encodes word importance or frequency—the dot product or Euclidean distance may be more appropriate. Always choose your similarity metric based on what your vectors encode.

3.5.2 Connection to Normalized Vectors

There is an elegant relationship between cosine similarity and the dot product of unit vectors. If $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ are the L2-normalized versions of $\mathbf{x}$ and $\mathbf{y}$, then:

$$\text{cos_sim}(\mathbf{x}, \mathbf{y}) = \hat{\mathbf{x}} \cdot \hat{\mathbf{y}}$$

This means that **for normalized vectors, cosine similarity and the dot product are identical**. This is why many neural networks normalize their embeddings before computing similarity—it converts the cheap dot product into the semantically meaningful cosine similarity.

Key Insight:

Normalize Once, Dot Product Forever In practice, many NLP systems pre-normalize all word vectors to unit length during a preprocessing step. After normalization, computing cosine similarity reduces to a single dot product—an operation that modern hardware (GPUs, TPUs) can execute extremely efficiently. This trick is used extensively in information retrieval, recommendation systems, and nearest-neighbor search.

3.5.3 Applications of Cosine Similarity

Cosine similarity is ubiquitous in NLP and information retrieval:

1. **Word Similarity**: Measuring how semantically close two words are. For example, $\text{cos_sim}(\vec{\text{cat}}, \vec{\text{kitten}}) \approx 0.9$ while $\text{cos_sim}(\vec{\text{cat}}, \vec{\text{algebra}}) \approx 0.1$.
2. **Document Similarity**: Representing documents as the average (or sum) of their word vectors, then computing cosine similarity between document vectors.
3. **Information Retrieval**: Ranking search results by cosine similarity between the query vector and document vectors.
4. **Duplicate Detection**: Identifying near-duplicate texts by thresholding cosine similarity (e.g., flag pairs with similarity > 0.95).

3.5.4 Implementation in NumPy

```python
import numpy as np

a = np.array([3, 4])
b = np.array([4, 3])

# Method 1: Direct formula
cos_sim = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(f"Cosine Similarity: {cos_sim:.4f}")  # 0.9600

# Method 2: Normalize first, then dot product
a_hat = a / np.linalg.norm(a)
b_hat = b / np.linalg.norm(b)
```

```
cos_sim_normalized = np.dot(a_hat, b_hat)
print(f"Via normalized dot: {cos_sim_normalized:.4f}")  # 0.9600
```

3.6 Introduction to Linguistic Embeddings

With the mathematical foundations in place, we now turn to the central question of this chapter: how do we represent *words* as vectors? Natural Language Processing (NLP) requires a way to translate human language—words, sentences, and documents—into a numerical format that machine learning models can understand. This process is known as **Vector Representation** or **Word Embedding**.

An **embedding** is a dense vector representation in a high-dimensional space that mathematically encodes semantic relationships between linguistic entities. Unlike simple numerical IDs, embeddings position similar words closer together in the vector space, capturing their meaning, context, and relationships.

3.6.1 Why Do We Need Embeddings?

Theoretical representations of text must solve several linguistic challenges:

1. **Semantic Similarity**: Words like "King" and "Queen" or "Man" and "Woman" share relationship patterns that should be mathematically calculable.
2. **Synonyms and Antonyms**: The model should understand that "joyful" and "happy" are close, while "happy" and "sad" are distinct (though related by sentiment).
3. **Polysemy**: Words like "bank" (financial vs. river) require context to distinguish their meaning.
4. **Language Morphology**: Particularly in morphologically rich languages like **Arabic**, words change forms extensively (e.g., *kitab*, *kitaban*, *maktaba*). Embeddings help capture the shared essence of these variations.

3.7 One-Hot Encoding and Its Limitations

The simplest way to represent words numerically is **One-Hot Encoding**. In this method:

1. We create a vocabulary of size V.
2. Each word is assigned a unique index.
3. A word is represented as a binary vector of size V, with a '1' at its specific index and '0' everywhere else.

Example:

One-Hot Encoding

- Book (*Kitab*): $[1, 0, 0, 0, 0]$
- Library (*Maktaba*): $[0, 1, 0, 0, 0]$

3.7.1 Limitations

While simple, One-Hot Encoding makes "garbage in, garbage out" a reality for advanced tasks due to:

1. **Lack of Semantic Similarity (Orthogonality)**:
   - The dot product of any two one-hot vectors is always 0.

- The model perceives "car" and "automobile" as being just as different as "car" and "banana". There is no encoded relationship.

2. **High Dimensionality and Sparsity**:
   - For a language like Arabic, the vocabulary can easily exceed 1,000,000 unique surface forms.
   - This results in vectors with one million dimensions, mostly containing zeros, which is computationally inefficient and wasteful.
3. **No Context Sensitivity**:
   - The word *'Ayn* (Spring/Eye/Spy) in Arabic would have the exact same one-hot vector regardless of whether it appears in "Spring of water" or "Eye of the spy".

Key Insight:

One-Hot Vectors and the Dot Product Recall from Section 3.3 that the dot product measures alignment between vectors. For one-hot vectors, *every* pair of distinct words yields a dot product of exactly zero—they are all mutually orthogonal. This means the representation encodes absolutely no information about word similarity. A vocabulary of 50,000 words produces 50,000 mutually perpendicular directions in a 50,000-dimensional space, treating "happy" and "joyful" as no more related than "happy" and "carburetor." Dense embeddings solve this by mapping words into a much lower-dimensional space where similar words are *not* orthogonal.

3.8 Frequency-Based Embedding Models (Count-Based)

To address the lack of meaning in one-hot vectors, *Frequency-Based* models (or Count-Based) use the statistics of word occurrences to derive representations. The intuition is that **words appearing in similar contexts or documents tend to be related**.

3.8.1 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF assigns weights to words based on their importance in a document relative to the entire corpus.

- **TF (Term Frequency)**: Measures how often a word appears in a specific document.

$$TF(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total words in } d}$$

- **IDF (Inverse Document Frequency)**: Measures how informative a word is across the corpus. It penalizes common words like "the" or "is".

$$IDF(t) = \log\left(\frac{N}{df_t}\right)$$

  Where N is total documents and df_t is the number of documents containing term t.

Example:

TF-IDF Application In a document about "Cats", the word "cat" has high TF and high IDF (if "cat" isn't in every other document), resulting in a high TF-IDF score. Common words get a score near 0.

3.8.2 Limitations of Count-Based Models

While better than One-Hot, TF-IDF still suffers from:

- **Sparsity**: Vectors are still the size of the vocabulary.
- **Limited Semantics**: It captures "importance" but not necessarily "meaning" or "similarity" in a way that supports vector arithmetic (e.g., analogies).

3.9 Prediction-Based Embeddings (Neural Probabilistic)

The modern revolution in NLP came with **Prediction-Based** embeddings. Instead of counting words, we **train a neural network** to predict a word given its context (or vice versa). The "side effect" of this training is a dense, low-dimensional vector (embedding) for each word.

3.9.1 Word2Vec Algorithms

Proposed by Mikolov et al. at Google (2013), Word2Vec is efficient because it uses a shallow neural network with a single hidden layer. It comes in two architectures:

1. Continuous Bag of Words (CBOW)

CBOW predicts the **target word** w_t based on the **context words** $(w_{t-m}, \ldots, w_{t+m})$.

- **Input**: One-hot vectors of context words.
- **Hidden Layer**: Averages the vectors of the context words.
- **Output**: Softmax probability distribution over the vocabulary.
- **Best for**: Frequent words and faster training.

2. Skip-Gram

Skip-Gram predicts the **context words** $(w_{t-m}, \ldots, w_{t+m})$ given the **target word** w_t.

- **Input**: One-hot vector of the target word.
- **Hidden Layer**: No averaging; direct projection.
- **Output**: Multiple Softmax classifiers (one for each context position).
- **Best for**: Rare words and smaller datasets.

3.9.2 Technical Deep Dive: The Skip-Gram Model

Let's trace exactly how the Skip-Gram model learns.

Architecture

The model consists of three layers:

1. **Input Layer**: A one-hot vector x of size V (Vocabulary size).
2. **Hidden Layer**: A weight matrix W of size $V \times N$, where N is the embedding dimension (e.g., 300). There is **no activation function** here. The hidden state h is simply:

   $$h = x^T W$$

   Since x is one-hot, this operations effectively **selects the k-th row** of W, corresponding to the word's embedding.
3. **Output Layer**: A second weight matrix W' of size $N \times V$. The output is a score vector $u = hW'$. We apply the **Softmax** function to get probabilities:

   $$P(w_O|w_I) = \text{softmax}(u) = \frac{\exp(u_{j^*})}{\sum_{j=1}^{V} \exp(u_j)}$$

Objective Function

We want to maximize the probability of predicting the correct context words given the center word over the entire corpus T.

$$\text{Maximize } J(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j}|w_t;\theta)$$

Loss Function

In practice, we minimize the **Negative Log Likelihood** (Cross-Entropy Loss):

$$E = -\log L(\theta) = -\sum_{t=1}^{T} \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j}|w_t)$$

Where:

$$\log P(w_O|w_I) = {v'_{w_O}}^T v_{w_I} - \log \sum_{w=1}^{V} \exp({v'_w}^T v_{w_I})$$

Here, v_{w_I} is the input vector (from W) and v'_{w_O} is the output context vector (from W').

3.9.3 Training Mechanics: How It Actually Learns

Since the denominator (sum over all V words) is computationally expensive used in standard Softmax, Word2Vec typically uses optimization approximation:

- **Negative Sampling**: Instead of updating all V weights, we update the correct word (positive sample) and a small number (e.g., 5-20) of incorrect words (negative samples).
- **Backpropagation**:
  1. Compute error $e_j = y_j - t_j$ (difference between predicted prob and actual 0/1).
  2. Backpropagate error to update W' (Output/Context embeddings).
  3. Backpropagate validation to update W (Input/Word embeddings).

Example:

Training Computation Example **Setup:**

- **Vocabulary** ($V = 5$): ["The", "cat", "sat", "on", "mat"]
- **Dimension** ($N = 3$)
- **Task**: Target = "cat" (Index 1), Context = "sat" (Index 2)

Execution Flow:

1. **Input Construction**: Create a one-hot vector for the target word "cat":

$$x = [0, 1, 0, 0, 0]$$

2. **Projection (Lookup)**: Multiply x by the Input Matrix W (5×3). This effectively selects the 2nd row (the embedding for "cat"):

$$h = x^T W = v_{cat} = [0.5, 0.1, -0.4]$$

3. **Context Scoring**: Multiply the hidden vector h by the Output Matrix W' (3×5). This produces a raw score (logit) for every word in the vocabulary:

$$u = v_{cat} \cdot W' = [1.2, \mathbf{3.5}, 0.2, -0.1, 0.5]$$

Note: Index 2 ("sat") has the highest score (3.5).

4. **Probability Conversion**: Apply the Softmax function to vector u to convert scores into normalized probabilities (summing to 1).

$$P(\text{"sat"}|\text{"cat"}) = \frac{e^{3.5}}{\sum e^{u_j}}$$

5. **Error Calculation**: Compare the predicted probability distribution with the actual one-hot vector for "sat". If the probability for "sat" is high, loss is low. If "The" had a higher score, gradients would flow back to adjust v_{cat} and v'_{sat} to align meaningful co-occurrences.

3.9.4 Linear Substructures (Analogies)

One of the most famous properties of Word2Vec is vector arithmetic.

$$\vec{\text{King}} - \vec{\text{Man}} + \vec{\text{Woman}} \approx \vec{\text{Queen}}$$

This demonstrates that the model has learned concepts like "gender" as a consistent direction in the vector space.

Key Insight:

Vector Arithmetic and the Math from Sections 3.1–3.5 The analogy $\vec{\text{King}} - \vec{\text{Man}} + \vec{\text{Woman}} \approx \vec{\text{Queen}}$ is a direct application of the vector addition and subtraction operations we studied in Section 3.2. The subtraction $\vec{\text{King}} - \vec{\text{Man}}$ isolates the "royalty" direction (removing the "male" component), and adding $\vec{\text{Woman}}$ translates along the "female" direction. The result is found using *cosine similarity* (Section 3.5) to locate the nearest word vector—in this case, "Queen." Without the mathematical foundations of Sections 3.1–3.5, these operations would be impossible to formalize or compute.

3.10 Global Vectors (GloVe)

While Word2Vec relies on local window-based learning (scanning the corpus one window at a time), **GloVe (Global Vectors)** combines the benefits of count-based matrix factorization and prediction-based methods.

3.10.1 The Core Intuition: Probability Ratios

The authors of GloVe (Pennington et al., Stanford 2014) argued that the ratio of co-occurrence probabilities is the key to encoding meaning.

Let $P(k|w)$ be the probability that word k appears in the context of word w. Consider the concept of thermodynamics with two focus words: **Ice** and **Steam**.

1. **Solid**: Related to "Ice" but not "Steam".
2. **Gas**: Related to "Steam" but not "Ice".
3. **Water**: Related to both.
4. **Fashion**: Related to neither.

Prob. k	**solid**	**gas**	**water**	**fashion**
$P(k\|\text{ice})$	High	Low	High	Low
$P(k\|\text{steam})$	Low	High	High	Low
Ratio $\frac{P(k\|\text{ice})}{P(k\|\text{steam})}$	**Large (>1)**	**Small (<1)**	$\approx$ **1**	$\approx$ **1**

Table 3.3: Co-occurrence Probabilities and Ratios

The **Ratio** is robust:

- A large ratio implies the word differentiates "Ice" from "Steam" (it correlates with Ice).
- A small ratio implies it correlates with "Steam".
- A ratio near 1 implies the word is either relevant to both (Water) or irrelevant to both (Fashion).

GloVe enforces that the dot product of word vectors equals the logarithm of their co-occurrence probability:

$$w_i^T w_k \approx \log P(k|i) = \log X_{ik} - \log X_i$$

3.10.2 The Mathematical Model

To learn these relationships, GloVe optimizes a weighted Least Squares objective function. We want the dot product of two word vectors w_i and w_j to equal the log count of their co-occurrence X_{ij}.

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

Where:

- X_{ij}: The number of times word j appears in the context of word i (Global Count).
- $w_i, \tilde{w}_j$: The word and context embeddings.
- $b_i, \tilde{b}_j$: Bias terms for each word.
- $f(X_{ij})$: A weighting function. It prevents rare words (low X_{ij}) from introducing noise and caps the influence of extremely frequent words (e.g., "the").

3.10.3 Training Algorithm

Unlike Word2Vec which iterates over the raw text, GloVe iterates over the **Non-Zero elements** of the co-occurrence matrix.

1. **Matrix Construction**: Scan the corpus once to populate the global matrix X.
   - Entry $X_{ij} = 10$ means "apple" and "eat" appeared together 10 times in the entire corpus.
2. **Initialization**: Randomly initialize word vectors w and context vectors $\tilde{w}$.
3. **Optimization**: For every non-zero pair (i, j) in matrix X:
   - Compute predicted log-count: $\hat{y} = w_i^T \tilde{w}_j + b_i + \tilde{b}_j$.
   - Compute error: $(\hat{y} - \log X_{ij})^2$.
   - Update vectors w_i and $\tilde{w}_j$ using Gradient Descent to minimize this error.

This approach is computationally efficient because the size of X grows with the vocabulary size squared, but the number of *non-zero* entries grows much slower (due to language sparsity).

3.11 FastText: Subword Information

Standard Word2Vec and GloVe treat each word (e.g., "eat" and "eating") as completely distinct atomic entities. This leads to two major problems:

1. **Morphology Ignorance**: The model ignores the internal structure of words (roots, prefixes, suffixes).
2. **Out-Of-Vocabulary (OOV) Failure**: If a word wasn't seen during training, the model cannot assign it a vector.

FastText (Bojanowski et al., Facebook AI Research 2017) solves this by representing a word not as a single vector, but as a **bag of character n-grams**.

3.11.1 The n-gram Representation

Instead of learning one parameter vector v_w for word w, FastText breaks the word into character n-grams (sub-sequences of length n). It adds special boundary symbols `<` and `>` to distinguish prefixes and suffixes.

Example:

Breaking down "apple" Consider the word **"apple"** with $n = 3$ (trigrams).

1. Add boundaries: `<apple>`
2. Generate 3-grams:
   - `<ap` (Prefix)
   - `app`
   - `ppl`
   - `ple`
   - `le>` (Suffix)
3. **Special Sequence**: The whole word `<apple>` is also included to capture the specific meaning of the full term.

3.11.2 Vector Generation

The final vector for the word "apple" is simply the **sum** of the vectors of its constituent n-grams.

$$\vec{v}_{apple} = \vec{z}_{<ap} + \vec{z}_{app} + \vec{z}_{ppl} + \vec{z}_{ple} + \vec{z}_{le>} + \vec{z}_{<apple>}$$

Where $\vec{z}_g$ is the learned vector for the n-gram g. This implies that the word "apple" shares the vector for `app` with "application" and `le>` with "table", ensuring shared morphological knowledge.

3.11.3 Handling Out-Of-Vocabulary (OOV) Words

This is FastText's superpower. Suppose the user types "eating" but the model has only seen "eat" and "reading" in training.

1. Word2Vec would fail (return `[UNK]`).
2. FastText succeeds by combining known parts:

   $$\vec{v}_{\text{eating}} \approx \vec{z}_{<ea} + \vec{z}_{eat} + \vec{z}_{ing} + \vec{z}_{ng>}$$

   Since it learned the vector for the suffix `ing` (from "reading", "singing") and the root `eat`, it constructs a meaningful vector for "eating" on the fly.

3.11.4 Example: Arabic Morphology

For morphologically rich languages like Arabic, FastText is essential.

- **Word**: *kitaban* (Two books).
- **Components**:

$$< ki, kit, ita, tab, aba, ban, an >$$

- **Shared Root**: The n-grams `kit`, `ita`, `tab` are shared with the word *kitab* (Book). The suffix `an>` indicates the Dual form.
- **Benefit**: The model understands that *kitaban* is related to *kitab* and is a dual noun, even if *kitaban* appears only once in the corpus.

3.12 Conclusion

This chapter has built a complete pipeline from raw mathematical objects to learned word representations. We began with the foundational concepts of vectors, vector operations, dot products, normalization, and cosine similarity—the mathematical toolkit required to work with any embedding. We then surveyed the evolution of word representation, from discrete symbols (One-Hot) to statistical weights (TF-IDF) and finally to dense, semantic vectors (Word2Vec, GloVe, FastText).

Concept / Model	Type	Key Feature	Best Use Case
Vectors & Norms	Math Foundation	Direction & magnitude	All ML representations
Dot Product	Math Operation	Alignment measurement	Similarity, attention
Cosine Similarity	Math Metric	Magnitude-independent similarity	Comparing embeddings
One-Hot	Symbolic	Sparse, Orthogonal	Small, closed vocabs
TF-IDF	Count-Based	Word Importance	Search/Keywords
Word2Vec	Prediction	Semantic Analogies	General Semantic tasks
GloVe	Hybrid	Global Co-occurrence	Global Correlations
FastText	Subword	Character n-grams	**Morphologically Rich (Ar**

Table 3.4: Comparison of Mathematical Foundations and Embedding Models

The mathematical operations from Sections 3.1–3.5 are not merely theoretical prerequisites; they are the *active ingredients* in every embedding model. The dot product drives Skip-Gram's scoring function and GloVe's objective. Cosine similarity is the standard metric for evaluating embedding quality. Vector arithmetic enables the celebrated analogy tests. Normalization ensures fair comparisons across words of different frequencies.

For modern Large Language Models (LLMs), these concepts form the foundation of the initial embedding layers, allowing Transformers to process deeper contextual relationships. In the next chapter, we will see how Transformers build upon these static embeddings with **positional encoding** and **self-attention**, enabling truly context-dependent representations where the same word receives different vectors depending on its surrounding sentence.

4

From Word Embedding to Contextual Embedding

4.1 Introduction — The Limitation of Static Embeddings

In the previous chapters, we learned how to convert words into dense numerical vectors called **embeddings**. Methods like Word2Vec and GloVe assign each word in the vocabulary a single, fixed vector — regardless of context. The word "bank" always receives the same embedding whether it appears in "river bank" or "bank account." This is a fundamental problem.

Warning:

The Static Embedding Trap Consider the following two sentences:

1. "I deposited money at the **bank**."
2. "The children played by the river **bank**."

In Word2Vec or GloVe, the word "bank" maps to the *same* vector in both cases. The embedding cannot distinguish between a *financial institution* and a *riverside slope*. This is because static embeddings encode **type-level** information (one vector per word type), not **token-level** information (one vector per word occurrence in context).

What we need is a representation that changes depending on the surrounding words — a **contextual embedding**. Given the sentence "I deposited money at the bank," the model should produce a vector for "bank" that is close to "finance," "account," and "deposit." Given "The children played by the river bank," the same word should yield a vector close to "shore," "water," and "nature."

This chapter traces the intellectual journey from static embeddings to contextual embeddings. We will cover:

1. **Text pre-processing** — how raw text is prepared for sequence models (Section 4.2).
2. **Feed-forward networks** — a quick recap to understand why they cannot handle sequences (Section 4.3).
3. **Recurrent Neural Networks (RNNs)** — the first architecture that introduces *memory* (Section 4.4).
4. **Activation functions** — why different nonlinearities serve different purposes (Section 4.5).
5. **Backpropagation Through Time** — how RNNs are trained (Section 4.6).
6. **The vanishing gradient problem** — the Achilles' heel of vanilla RNNs (Section 4.7).
7. **LSTM** — the gated architecture that solves vanishing gradients (Section 4.8).
8. **GRU** — a streamlined alternative to LSTM (Section 4.9).
9. **The complete picture** — how all of this leads to Transformers (Section 4.10).

Key Insight:

The Evolution of Word Representations The history of word representations in NLP follows a clear trajectory of increasing expressiveness:

- **One-hot vectors** (sparse, no semantics) →
- **Word2Vec / GloVe** (dense, semantic, but static) →
- **RNN hidden states** (contextual, but limited memory) →
- **LSTM / GRU hidden states** (contextual, longer memory) →
- **Transformer outputs** (contextual, unlimited memory, parallelizable).

Each step overcomes a specific limitation of the previous one. This chapter covers the middle three steps.

4.2 Text Pre-Processing for Sequence Models

Before feeding text into any neural network, we must convert raw strings into numerical tensors of uniform shape. This section describes the standard six-step pipeline used by virtually all sequence models.

Definition: Text Pre-Processing Pipeline

The standard pipeline for preparing text for a sequence model consists of six stages:

1. **Clean**: Remove noise (HTML tags, special characters, extra whitespace).
2. **Normalize / Stem**: Reduce words to base forms (e.g., "running" → "run").
3. **Tokenize**: Split text into tokens (words or subwords).
4. **Index**: Map each token to an integer using a vocabulary lookup table.
5. **Pad**: Make all sequences the same length by appending a special `<PAD>` token.
6. **Embed**: Convert each integer index into a dense vector via an embedding matrix.

4.2.1 Why Padding is Necessary

Neural networks process data in **batches**. A batch is a matrix where each row is one training example. For this matrix to exist, every row must have the same number of columns — that is, every sequence must have the same length.

Consider a batch of three sentences:

- "I love deep learning" (4 tokens)
- "AI is great" (3 tokens)
- "Neural networks are powerful tools" (5 tokens)

We pad all sequences to the length of the longest one (5 tokens):

- `[I, love, deep, learning, <PAD>]`
- `[AI, is, great, <PAD>, <PAD>]`
- `[Neural, networks, are, powerful, tools]`

4.2.2 Masking: Ignoring Padding Tokens

Padding tokens carry no meaning. If the model attends to them, it will learn noise. A **mask** is a binary tensor that marks which positions are real tokens (1) and which are padding (0). The model uses this mask to zero out contributions from padded positions during computation.

For the batch above, the mask would be:

$$\text{mask} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Example:

Complete Pre-Processing Pipeline in PyTorch The following code demonstrates the full six-step pipeline:

```python
import torch
import torch.nn as nn
import re

# === Step 1: Clean ===
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)  # keep only letters and spaces
    return text.strip()

# === Step 2: Normalize (simple stemming) ===
def simple_stem(word):
    if word.endswith('ing'):
        return word[:-3]
    if word.endswith('ly'):
        return word[:-2]
    return word

# === Step 3 & 4: Tokenize and Index ===
vocab = {'<PAD>': 0, '<UNK>': 1, 'i': 2, 'love': 3, 'deep': 4,
         'learn': 5, 'ai': 6, 'is': 7, 'great': 8}

def tokenize_and_index(text, vocab):
    words = text.split()
    stemmed = [simple_stem(w) for w in words]
    return [vocab.get(w, vocab['<UNK>']) for w in stemmed]

# === Step 5: Pad ===
def pad_sequences(sequences, pad_idx=0):
    max_len = max(len(s) for s in sequences)
    padded = [s + [pad_idx] * (max_len - len(s)) for s in sequences]
    return torch.tensor(padded)

# === Step 6: Embed ===
embedding = nn.Embedding(num_embeddings=len(vocab), embedding_dim=16,
                         padding_idx=0)

# Full pipeline
sentences = ["I love deep learning!", "AI is great."]
cleaned = [clean_text(s) for s in sentences]
indexed = [tokenize_and_index(s, vocab) for s in cleaned]
padded = pad_sequences(indexed)      # shape: (2, 4)
mask = (padded != 0).long()          # shape: (2, 4)
embedded = embedding(padded)         # shape: (2, 4, 16)
```

```python

print(f"Padded:   {padded}")
print(f"Mask:     {mask}")
print(f"Embedded: {embedded.shape}")
```

4.3 The Perceptron and Feed-Forward Networks (Recap)

Before introducing recurrent networks, let us briefly recall the building block of all neural networks: the **perceptron**.

4.3.1 The Single Perceptron

A perceptron takes an input vector $\mathbf{x} \in \mathbb{R}^m$, computes a weighted sum, adds a bias, and passes the result through an activation function:

$$y = \sigma\left(\sum_{i=1}^{m} w_i x_i + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \tag{4.1}$$

where $\mathbf{w} \in \mathbb{R}^m$ is the weight vector, $b \in \mathbb{R}$ is the bias, and σ is an activation function (e.g., sigmoid, ReLU).

4.3.2 Multi-Layer Perceptron (MLP)

By stacking multiple layers of perceptrons, we obtain a **multi-layer perceptron** (MLP) or **feed-forward neural network** (FFN):

$$\mathbf{h}_1 = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \tag{4.2}$$
$$\mathbf{h}_2 = \sigma(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2) \tag{4.3}$$
$$\hat{y} = \text{softmax}(\mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3) \tag{4.4}$$

An MLP is a **universal function approximator**: given enough hidden units, it can represent any continuous function. However, it has a critical limitation for language processing.

4.3.3 The Key Limitation: No Memory

Warning:

Feed-Forward Networks Cannot Handle Sequences An MLP processes each input **independently**. If we feed it the word "bank," it produces the same output regardless of whether the previous word was "river" or "savings." It has **no memory** of previous inputs.
This makes MLPs fundamentally unsuitable for sequential data like language, where the meaning of each word depends on the words that came before it. We need an architecture that can **carry information from one time step to the next**.

4.4 Recurrent Neural Networks (RNN)

4.4.1 The Core Idea

The key insight behind Recurrent Neural Networks (RNNs) is deceptively simple: **give the network a memory**. At each time step, the network receives two inputs — the current word and a summary of everything it has seen so far. This summary is called the **hidden state**.

Definition: Recurrent Neural Network

A Recurrent Neural Network (RNN) is a neural network that processes sequential data by maintaining a **hidden state** $\mathbf{h}_t$ that is updated at each time step t. The hidden state serves as the network's memory, encoding information about all previous inputs in the sequence.

At each time step t, the RNN performs two operations:

1. **Update the hidden state** by combining the previous hidden state $\mathbf{h}_{t-1}$ with the current input $\mathbf{x}_t$.
2. **Produce an output** based on the updated hidden state $\mathbf{h}_t$.

The recurrence relation is:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}) \tag{4.5}$$

Key Insight:

Weight Sharing Across Time Steps A crucial property of RNNs is that the same weight matrices $\mathbf{W}_h$, $\mathbf{W}_x$, and bias $\mathbf{b}$ are used at *every* time step. This is called **weight sharing** or **parameter tying**. It means:

- The network applies the *same transformation* at each step, regardless of sequence length.
- The model can process sequences of arbitrary length without increasing the number of parameters.
- The network learns a *general rule* for updating its memory, not a position-specific rule.

This is analogous to how a convolutional filter is shared across spatial positions in a CNN.

Conceptually, an RNN can be "unrolled" across time. Imagine making a copy of the network for each time step and connecting the hidden state of each copy to the next. The unrolled network looks like a deep feed-forward network with shared weights:

$$\mathbf{x}_1 \to \boxed{\text{RNN}} \to \mathbf{h}_1 \to \boxed{\text{RNN}} \to \mathbf{h}_2 \to \boxed{\text{RNN}} \to \mathbf{h}_3$$

Each box is the *same* network with the *same* parameters, applied at successive time steps.

4.4.2 Mathematical Formulation

Let us make the dimensions precise. At each time step t:

- **Input**: $\mathbf{x}_t \in \mathbb{R}^m$ (e.g., a word embedding of dimension m)
- **Previous hidden state**: $\mathbf{h}_{t-1} \in \mathbb{R}^n$ (hidden dimension n)
- **Input-to-hidden weights**: $\mathbf{W}_x \in \mathbb{R}^{n \times m}$
- **Hidden-to-hidden weights**: $\mathbf{W}_h \in \mathbb{R}^{n \times n}$
- **Bias**: $\mathbf{b} \in \mathbb{R}^n$

The hidden state update is:

$$\mathbf{h}_t = \tanh(\mathbf{W}_x\mathbf{x}_t + \mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{b}) \in \mathbb{R}^n \tag{4.6}$$

The output at time step t (e.g., for classification) is:

$$\mathbf{y}_t = \text{softmax}(\mathbf{W}_y\mathbf{h}_t + \mathbf{b}_y) \in \mathbb{R}^c \tag{4.7}$$

where $\mathbf{W}_y \in \mathbb{R}^{c\times n}$ maps the hidden state to the output space of c classes.

Key Insight:

Dimensionality Analysis Let us verify that the matrix dimensions are consistent. In Equation 4.5:

$$\underbrace{\mathbf{W}_x}_{n\times m}\underbrace{\mathbf{x}_t}_{m\times 1} + \underbrace{\mathbf{W}_h}_{n\times n}\underbrace{\mathbf{h}_{t-1}}_{n\times 1} + \underbrace{\mathbf{b}}_{n\times 1} = \underbrace{\mathbf{h}_t}_{n\times 1}$$

Each term produces an n-dimensional vector, so the addition is valid. The tanh is applied element-wise, preserving the dimension. Always verify matrix dimensions before implementing — mismatches are one of the most common bugs in deep learning code.

The initial hidden state $\mathbf{h}_0$ is typically initialized to the zero vector:

$$\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^n$$

4.4.3 Worked Example — Sentiment Analysis of "I like AI"

Let us trace through the computation of an RNN processing the sentence "I like AI" for binary sentiment classification. We use small matrices (3×3) for clarity.

Example:

RNN Forward Pass: "I like AI" **Setup:**

- Embedding dimension: $m = 3$
- Hidden dimension: $n = 3$
- Vocabulary: {"I": 0, "like": 1, "AI": 2}

Word embeddings (from a pre-trained or randomly initialized embedding matrix):

$$\mathbf{x}_{\text{I}} = \begin{bmatrix}1\\0\\0\end{bmatrix}, \quad \mathbf{x}_{\text{like}} = \begin{bmatrix}0\\1\\0\end{bmatrix}, \quad \mathbf{x}_{\text{AI}} = \begin{bmatrix}0\\0\\1\end{bmatrix}$$

Weight matrices (simplified for illustration):

$$\mathbf{W}_x = \begin{bmatrix}0.5 & 0.0 & 0.3\\0.1 & 0.7 & 0.0\\0.0 & 0.2 & 0.6\end{bmatrix}, \quad \mathbf{W}_h = \begin{bmatrix}0.2 & 0.0 & 0.1\\0.0 & 0.3 & 0.0\\0.1 & 0.0 & 0.4\end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix}0\\0\\0\end{bmatrix}$$

Initial hidden state: $\mathbf{h}_0 = [0, 0, 0]^\top$

Step 1: Process "I" ($t = 1$)

$$\mathbf{z}_1 = \mathbf{W}_x\mathbf{x}_{\text{I}} + \mathbf{W}_h\mathbf{h}_0 + \mathbf{b} = \begin{bmatrix}0.5\\0.1\\0.0\end{bmatrix} + \begin{bmatrix}0\\0\\0\end{bmatrix} = \begin{bmatrix}0.5\\0.1\\0.0\end{bmatrix}$$

$$\mathbf{h}_1 = \tanh(\mathbf{z}_1) = \begin{bmatrix} \tanh(0.5) \\ \tanh(0.1) \\ \tanh(0.0) \end{bmatrix} = \begin{bmatrix} 0.462 \\ 0.100 \\ 0.000 \end{bmatrix}$$

Step 2: Process "like" ($t = 2$)

$$\mathbf{z}_2 = \mathbf{W}_x \mathbf{x}_{\text{like}} + \mathbf{W}_h \mathbf{h}_1 + \mathbf{b}$$

$$\mathbf{W}_x \mathbf{x}_{\text{like}} = \begin{bmatrix} 0.0 \\ 0.7 \\ 0.2 \end{bmatrix}, \quad \mathbf{W}_h \mathbf{h}_1 = \begin{bmatrix} 0.2 \times 0.462 + 0.0 \times 0.100 + 0.1 \times 0.0 \\ 0.0 \times 0.462 + 0.3 \times 0.100 + 0.0 \times 0.0 \\ 0.1 \times 0.462 + 0.0 \times 0.100 + 0.4 \times 0.0 \end{bmatrix} = \begin{bmatrix} 0.092 \\ 0.030 \\ 0.046 \end{bmatrix}$$

$$\mathbf{z}_2 = \begin{bmatrix} 0.092 \\ 0.730 \\ 0.246 \end{bmatrix}, \quad \mathbf{h}_2 = \tanh(\mathbf{z}_2) = \begin{bmatrix} 0.092 \\ 0.623 \\ 0.241 \end{bmatrix}$$

Step 3: Process "AI" ($t = 3$)

$$\mathbf{W}_x \mathbf{x}_{\text{AI}} = \begin{bmatrix} 0.3 \\ 0.0 \\ 0.6 \end{bmatrix}, \quad \mathbf{W}_h \mathbf{h}_2 = \begin{bmatrix} 0.2 \times 0.092 + 0.1 \times 0.241 \\ 0.3 \times 0.623 \\ 0.1 \times 0.092 + 0.4 \times 0.241 \end{bmatrix} = \begin{bmatrix} 0.042 \\ 0.187 \\ 0.106 \end{bmatrix}$$

$$\mathbf{z}_3 = \begin{bmatrix} 0.342 \\ 0.187 \\ 0.706 \end{bmatrix}, \quad \mathbf{h}_3 = \tanh(\mathbf{z}_3) = \begin{bmatrix} 0.329 \\ 0.185 \\ 0.608 \end{bmatrix}$$

Classification: The final hidden state $\mathbf{h}_3 = [0.329, 0.185, 0.608]^\top$ is the sentence representation. For binary sentiment classification, we pass it through a linear layer and sigmoid:

$$p(\text{positive}) = \sigma(\mathbf{w}_y^\top \mathbf{h}_3 + b_y)$$

If $\mathbf{w}_y = [0.5, 0.3, 0.8]^\top$ and $b_y = -0.2$, then:

$$p(\text{positive}) = \sigma(0.5 \times 0.329 + 0.3 \times 0.185 + 0.8 \times 0.608 - 0.2) = \sigma(0.507) = 0.624$$

The model predicts a 62.4% probability of positive sentiment.

4.4.4 How RNNs Create Contextual Embeddings

Now we can revisit the "bank" problem from the introduction. Consider two sentences:

1. "I deposited money at the bank"
2. "I sat by the river bank"

In a static embedding model, the word "bank" has the same vector in both sentences. But in an RNN, the hidden state $\mathbf{h}_t$ at the position of "bank" encodes the *entire prefix* of the sentence up to that point.

- In Sentence 1: $\mathbf{h}_{\text{bank}}$ is influenced by "deposited," "money," and "at" — words associated with finance.
- In Sentence 2: $\mathbf{h}_{\text{bank}}$ is influenced by "sat," "river" — words associated with nature.

Therefore, the hidden state at the position of "bank" is **different** in the two sentences, even though the input embedding of "bank" is the same. The RNN has created a **contextual embedding** — a representation that depends on the surrounding words.

Key Insight:

From Static to Contextual: The Key Insight In a static embedding model, a word's representation is determined solely by its identity:

$$\text{static: } \mathbf{e}_{\text{bank}} = \text{Embed}(\text{“bank”})$$

In an RNN, a word's representation is determined by its identity *and* its entire preceding context:

$$\text{contextual: } \mathbf{h}_t = f(\text{“bank”}, \text{“I deposited money at the”})$$

The hidden state $\mathbf{h}_t$ at position t encodes the cumulative effect of all words from position 1 to position t. This is the fundamental advance of recurrent networks over static embeddings.

4.4.5 Sequential Model Architectures

RNNs are remarkably flexible. By varying how inputs and outputs are connected, we obtain architectures suited to different tasks:

1. **One-to-One**: A single input produces a single output. This is equivalent to a standard feed-forward network (e.g., image classification). No recurrence is needed.
2. **One-to-Many**: A single input produces a sequence of outputs. Example: **image captioning**, where a CNN encodes an image into a single vector, and an RNN decodes it into a sequence of words.
3. **Many-to-One**: A sequence of inputs produces a single output. Example: **sentiment analysis**, where the model reads an entire review and outputs a single label (positive/negative). Only the final hidden state $\mathbf{h}_T$ is used for prediction.
4. **Many-to-Many (Aligned)**: Each input produces a corresponding output. Example: **named entity recognition** (NER), where each word in a sentence receives a label (Person, Organization, Location, etc.).
5. **Many-to-Many (Unaligned / Encoder-Decoder)**: A variable-length input sequence produces a variable-length output sequence. Example: **machine translation**, where "I love cats" (3 words) might translate to "J'aime les chats" (3 words in French, but could be different in other languages). This requires an *encoder* RNN to compress the input into a fixed vector, and a *decoder* RNN to generate the output from that vector.

Key Insight:

The Encoder-Decoder Bottleneck In the many-to-many (encoder-decoder) architecture, the entire input sequence is compressed into a single fixed-size vector — the final hidden state of the encoder. This creates a severe **information bottleneck**: no matter how long the input sentence is, all of its meaning must be squeezed into one vector of dimension n. This is one of the key limitations that the **attention mechanism** (Chapter 5) was designed to solve.

4.5 Activation Functions

Before diving deeper into training and advanced architectures, we need to understand why different parts of a neural network use different activation functions. The choice is never arbitrary — each activation serves a specific mathematical purpose.

4.5.1 The Three Key Activation Functions

Hyperbolic Tangent (tanh)

The tanh function maps any real number to the range $(-1, +1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{4.8}$$

Its derivative is:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z) \tag{4.9}$$

Use case: Hidden state computations in RNNs and LSTMs. Because tanh outputs values in $(-1, +1)$, it can represent both *positive* and *negative* information. The hidden state can "vote" for or against certain features.

Sigmoid (σ)

The sigmoid function maps any real number to the range $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{4.10}$$

Its derivative is:

$$\frac{d}{dz}\sigma(z) = \sigma(z)(1 - \sigma(z)) \tag{4.11}$$

Use case: Gates in LSTMs and GRUs. Since the output is between 0 and 1, sigmoid functions act as **soft switches** — they control how much information flows through. A value of 0 means "block everything"; a value of 1 means "let everything through."

Rectified Linear Unit (ReLU)

The ReLU function is piecewise linear:

$$\mathrm{ReLU}(z) = \max(0, z) \tag{4.12}$$

Its derivative is:

$$\frac{d}{dz}\mathrm{ReLU}(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \tag{4.13}$$

Use case: Hidden layers in deep feed-forward networks and Transformers. ReLU is computationally cheap and does not suffer from saturation for positive inputs, making it ideal for deep networks.

4.5.2 Comparison

Function	Range	Formula	Derivative	Primary Use
tanh	$(-1, +1)$	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$	Hidden states
σ (sigmoid)	$(0, 1)$	$\frac{1}{1+e^{-z}}$	$\sigma(z)(1 - \sigma(z))$	Gates
ReLU	$[0, \infty)$	$\max(0, z)$	$\{0, 1\}$	FFN layers

Table 4.1: Comparison of activation functions used in sequence models.

Definition: Why tanh for Hidden States and Sigmoid for Gates

The choice of activation function in gated architectures like LSTM is deliberate:

- **tanh for hidden states**: The hidden state must represent information with both positive and negative polarity. For example, a sentiment model needs to encode both positive ("love", "great") and negative ("hate", "terrible") signals. The range $(-1, +1)$ provides this symmetry. Furthermore, tanh is zero-centered, which helps gradient flow during training.
- **Sigmoid for gates**: A gate must make a binary-like decision: how much to remember vs. how much to forget. The range $(0, 1)$ naturally represents a **proportion** — 0 means "completely forget" and 1 means "completely remember." Sigmoid provides a smooth, differentiable approximation of this binary choice.

4.6 Training RNNs — Backpropagation Through Time (BPTT)

Training an RNN requires computing how the loss function changes with respect to each weight in the network. Since the same weights are used at every time step, we must propagate gradients *backward through time* — through every step of the unrolled network.

4.6.1 The Unrolled Computation Graph

Consider an RNN processing a sequence of T tokens. The total loss is the sum of losses at each time step:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t \tag{4.14}$$

To compute the gradient of $\mathcal{L}$ with respect to the shared weight matrix $\mathbf{W}_h$, we must sum contributions from every time step:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_h} \tag{4.15}$$

4.6.2 The Gradient Chain Rule Through Time

At any time step t, the hidden state $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, which depends on $\mathbf{h}_{t-2}$, and so on. By the chain rule:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_h} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left(\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_h} \tag{4.16}$$

The critical term is the product of Jacobians:

$$\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \tag{4.17}$$

Each factor in this product is:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \mathrm{diag}(\tanh'(\mathbf{z}_j)) \cdot \mathbf{W}_h^\top \tag{4.18}$$

where $\mathrm{diag}(\tanh'(\mathbf{z}_j))$ is a diagonal matrix of the tanh derivatives at time step j. This product of matrices is at the heart of both the power and the weakness of RNNs.

4.7 The Vanishing Gradient Problem

The product of Jacobians in Equation 4.16 is the source of the most infamous problem in deep learning history: the **vanishing gradient problem**.

4.7.1 Mathematical Explanation

Consider the product:

$$\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \prod_{j=k+1}^{t} \operatorname{diag}(\tanh'(\mathbf{z}_j)) \cdot \mathbf{W}_h^\top \tag{4.19}$$

For simplicity, let us analyze the scalar case. If the network has a single hidden unit, then:

$$\prod_{j=k+1}^{t} \tanh'(z_j) \cdot w_h \tag{4.20}$$

Since $|\tanh'(z)| \leq 1$ for all z, each factor in the product has magnitude at most $|w_h|$. Over T time steps:

- If $|\tanh'(z) \cdot w_h| < 1$: the product **decays exponentially** $\rightarrow$ **vanishing gradients**.
- If $|\tanh'(z) \cdot w_h| > 1$: the product **grows exponentially** $\rightarrow$ **exploding gradients**.

Example:

Exponential Decay in Practice Suppose $|\tanh'(z) \cdot w_h| \approx 0.9$ at each time step. After 100 time steps:

$$0.9^{100} \approx 0.0000266$$

The gradient from time step 100 reaching time step 1 is attenuated by a factor of nearly 40,000. The network effectively **cannot learn** long-range dependencies.
Even with a more generous factor of 0.95:

$$0.95^{100} \approx 0.0059$$

The gradient is still reduced by a factor of 170. Information from the distant past is essentially invisible during training.

4.7.2 Intuitive Example

Consider the sentence:

> "Despite the cat's initial fear of the thunderstorm, which had been particularly loud that evening and had rattled the windows, over the following months the cat and the dog became _____."

The correct word is "friends" (or "best friends"). To predict this, the model must connect "the cat" (near the beginning) with "became" (near the end) across a span of roughly 25 words. With vanishing gradients, the gradient signal from "became friends" cannot propagate back far enough to learn that "cat" is relevant.

4.7.3 Gradient Clipping

Exploding gradients can be mitigated by a simple technique called **gradient clipping**: if the gradient norm exceeds a threshold θ, we rescale it:

$$\mathbf{g} \leftarrow \frac{\theta}{\|\mathbf{g}\|} \cdot \mathbf{g} \quad \text{if } \|\mathbf{g}\| > \theta \tag{4.21}$$

This prevents catastrophic updates but does not solve the fundamental problem.

Warning:

Gradient Clipping Does Not Solve Vanishing Gradients Gradient clipping addresses exploding gradients by capping their magnitude. However, **vanishing gradients** cannot be fixed by simply scaling up small gradients — the information has been destroyed by the repeated multiplication. The gradient is not just small; it is uninformative. Solving vanishing gradients requires a fundamentally different architecture — one where gradients can flow through the network without being multiplied at every step. This is exactly what the LSTM provides.

4.8 LSTM — Long Short-Term Memory

The Long Short-Term Memory (LSTM) network, proposed by Hochreiter and Schmidhuber in 1997, is arguably the most important architectural innovation in the history of recurrent neural networks. It solves the vanishing gradient problem by introducing a dedicated **memory cell** and **gates** that control the flow of information.

4.8.1 The Core Idea

The fundamental insight of the LSTM is to create a separate memory channel — the **cell state** $\mathbf{C}_t$ — through which information can flow across many time steps with minimal modification. Think of the cell state as a **conveyor belt** running through the entire sequence. Information can be placed on the belt, read from the belt, or removed from the belt, but the belt itself moves forward undisturbed.

Definition: LSTM Cell

An LSTM cell maintains two state vectors at each time step:

- $\mathbf{C}_t \in \mathbb{R}^n$: the **cell state** — long-term memory.
- $\mathbf{h}_t \in \mathbb{R}^n$: the **hidden state** — short-term / working memory and output.

The cell state is regulated by three **gates**, each controlled by a sigmoid function:

1. **Forget gate** $\mathbf{f}_t$: decides what to *remove* from the cell state.
2. **Input gate** $\mathbf{i}_t$: decides what new information to *add* to the cell state.
3. **Output gate** $\mathbf{o}_t$: decides what to *expose* as the hidden state.

In a vanilla RNN, information is transformed by **matrix multiplication** at every time step, which causes gradients to vanish or explode. In an LSTM, the cell state is updated by **addition**, which allows gradients to flow through unchanged. This single architectural decision is what makes LSTMs work.

4.8.2 The Forget Gate

The forget gate decides which information from the previous cell state $\mathbf{C}_{t-1}$ should be discarded. It looks at the previous hidden state $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$, and outputs a vector of values between 0 and 1:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \tag{4.22}$$

where $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ denotes the concatenation of $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ into a single vector of dimension $n + m$.

Each element of $\mathbf{f}_t$ acts as a **valve**:

- $f_t^{(i)} \approx 0$: forget the i-th component of the cell state (valve closed).
- $f_t^{(i)} \approx 1$: keep the i-th component unchanged (valve open).

Example:

Forget Gate in Action Consider a language model processing the text: "The cat sat on the mat. The dog ran in the park."
When the model reaches the period after "mat," the forget gate should learn to *forget* information about the cat (the old subject), because a new sentence with a new subject ("the dog") is beginning. The forget gate outputs values close to 0 for the memory cells that stored "cat"-related information, effectively erasing them to make room for "dog."

4.8.3 The Input Gate and Candidate

The input gate decides which new information to store in the cell state. This involves two computations:

1. The **input gate** determines *how much* of the new information to add:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \tag{4.23}$$

2. The **candidate cell state** determines *what* new information is available:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \tag{4.24}$$

Notice the different activation functions: **sigmoid** for the gate (outputs values in $[0, 1]$ representing proportions) and **tanh** for the candidate (outputs values in $[-1, +1]$ representing actual content with polarity).

4.8.4 The Cell State Update

This is the most important equation in the LSTM — the one that solves the vanishing gradient problem:

$$\boxed{\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t} \tag{4.25}$$

where $\odot$ denotes element-wise (Hadamard) multiplication.
Let us read this equation term by term:

- $\mathbf{f}_t \odot \mathbf{C}_{t-1}$: **Selectively forget** parts of the old memory. If $f_t^{(i)} = 1$, the old memory is fully preserved. If $f_t^{(i)} = 0$, it is erased.

- $\mathbf{i}_t \odot \tilde{\mathbf{C}}_t$: **Selectively add** new information. The input gate $\mathbf{i}_t$ controls how much of each candidate value is actually written.
- The **addition** (+) between the two terms is the key: gradients flow through addition with a derivative of 1. No matrix multiplication, no vanishing.

Key Insight:

Why Addition Solves Vanishing Gradients Compare the gradient flow in a vanilla RNN vs. an LSTM:

Vanilla RNN:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(\tanh'(\mathbf{z}_t)) \cdot \mathbf{W}_h^\top$$

This involves a matrix multiplication at every step, causing exponential decay/growth.

LSTM cell state:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \text{diag}(\mathbf{f}_t)$$

The gradient of $\mathbf{C}_t$ with respect to $\mathbf{C}_{t-1}$ is simply the forget gate vector $\mathbf{f}_t$. If $\mathbf{f}_t \approx \mathbf{1}$ (the gate is "open"), the gradient flows through **unchanged**. There is no matrix multiplication — just element-wise scaling by a value close to 1. This is the mathematical mechanism by which LSTMs preserve long-range dependencies.

4.8.5 The Output Gate

The output gate controls what information from the cell state is exposed as the hidden state (which serves as both the output and the short-term memory passed to the next step):

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \tag{4.26}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \tag{4.27}$$

The cell state $\mathbf{C}_t$ is passed through tanh (to squash values to $[-1, +1]$) and then filtered by the output gate. This means the cell state can store information that is not immediately relevant to the output but might be needed later.

Example:

Complete LSTM Equations Summary At each time step t, the LSTM computes:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(Forget gate)} \tag{4.28}$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(Input gate)} \tag{4.29}$$
$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad \text{(Candidate)} \tag{4.30}$$
$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad \text{(Cell update)} \tag{4.31}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(Output gate)} \tag{4.32}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad \text{(Hidden state)} \tag{4.33}$$

Each of the four weight matrices $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_C, \mathbf{W}_o$ has shape $(n \times (n+m))$, where n is the hidden dimension and m is the input dimension.

Total parameters per LSTM layer: $4 \times [n(n+m)+n] = 4n^2 + 4nm + 4n$.

4.8.6 LSTM Variants

Several modifications to the standard LSTM have been proposed:

Peephole Connections (Gers & Schmidhuber, 2000)

In the standard LSTM, gates can see $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$, but not the cell state $\mathbf{C}_{t-1}$ directly. Peephole connections add the cell state as an additional input to the gates:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{W}_{pf} \odot \mathbf{C}_{t-1} + \mathbf{b}_f) \tag{4.34}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{W}_{pi} \odot \mathbf{C}_{t-1} + \mathbf{b}_i) \tag{4.35}$$

This allows the gates to make decisions based on the actual stored memory, not just the filtered output.

Coupled Forget-Input Gates

Instead of independently deciding what to forget and what to add, we can couple the two decisions:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + (1 - \mathbf{f}_t) \odot \tilde{\mathbf{C}}_t \tag{4.36}$$

Here, $\mathbf{i}_t = 1 - \mathbf{f}_t$. This enforces the constraint that whenever we forget something, we must add something in its place, and vice versa. This reduces the number of parameters and can improve regularization.

4.8.7 Limitations of LSTMs

Despite their success, LSTMs have significant drawbacks:

1. **Complexity**: Each LSTM cell has four sets of weight matrices (forget, input, candidate, output). For a hidden size of $n = 1024$ and input size of $m = 512$, this means $4 \times 1024 \times (1024 + 512) = 6{,}291{,}456$ parameters per layer — just for the recurrent weights.
2. **Training time**: The four matrix multiplications per time step make LSTMs approximately 4 times slower than vanilla RNNs.
3. **Sequential computation**: Because $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, each time step must be computed *after* the previous one. This means LSTMs **cannot be parallelized** across time steps. On modern GPUs with thousands of cores, this is a severe bottleneck.
4. **Limited long-range memory**: While LSTMs are much better than vanilla RNNs at capturing long-range dependencies, they still struggle with sequences longer than a few hundred tokens. The cell state, though protected by gates, still undergoes some information loss over very long distances.

These limitations directly motivate the development of the GRU (a simpler alternative) and ultimately the Transformer (a fundamentally different approach).

4.9 GRU — Gated Recurrent Unit

The Gated Recurrent Unit (GRU), introduced by Cho et al. in 2014, simplifies the LSTM by merging the cell state and hidden state into a single state vector and reducing the number of gates from three to two.

4.9.1 Motivation — Simplifying LSTM

The GRU asks: do we really need a separate cell state and hidden state? Can we achieve similar performance with fewer gates and fewer parameters? The answer, empirically, is often yes.

Definition: Gated Recurrent Unit

A GRU maintains a single state vector $\mathbf{h}_t \in \mathbb{R}^n$ and uses two gates:

1. **Reset gate $\mathbf{r}_t$**: controls how much of the past state to use when computing the candidate.
2. **Update gate $\mathbf{z}_t$**: controls the trade-off between keeping the old state and adopting the new candidate. It combines the roles of the LSTM's forget and input gates.

4.9.2 The Reset Gate

The reset gate determines how much of the previous hidden state $\mathbf{h}_{t-1}$ is relevant for computing the new candidate:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \tag{4.37}$$

When $\mathbf{r}_t \approx \mathbf{0}$, the GRU ignores the previous state entirely and computes the candidate based on the current input alone — effectively "resetting" its memory. When $\mathbf{r}_t \approx \mathbf{1}$, the full previous state is used.

4.9.3 The Update Gate

The update gate controls how much of the new candidate to blend with the old state:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \tag{4.38}$$

This single gate replaces *both* the forget gate and the input gate of the LSTM. The key insight is:

- $z_t \approx 1$: keep the old state $\mathbf{h}_{t-1}$ (like LSTM's $f_t \approx 1, i_t \approx 0$).
- $z_t \approx 0$: replace with the new candidate (like LSTM's $f_t \approx 0, i_t \approx 1$).

4.9.4 Hidden State Computation

First, the **candidate hidden state** is computed using the reset gate to selectively mask the previous state:

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W} \cdot [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}) \tag{4.39}$$

Then, the final hidden state is an **interpolation** between the old state and the new candidate, controlled by the update gate:

$$\boxed{\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t} \tag{4.40}$$

Key Insight:

The GRU Update as Linear Interpolation Equation 4.40 is a **convex combination** (linear interpolation) of $\mathbf{h}_{t-1}$ and $\tilde{\mathbf{h}}_t$, controlled element-wise by $\mathbf{z}_t$. This is elegant:

- When $z_t^{(i)} = 0$: the i-th component is copied directly from the old state. The gradient flows through unchanged, preventing vanishing gradients for that dimension.
- When $z_t^{(i)} = 1$: the i-th component is entirely replaced by the candidate.
- For intermediate values: the result is a smooth blend.

This interpolation mechanism is analogous to the LSTM cell state update ($\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$), but with the constraint $\mathbf{i}_t = 1 - \mathbf{f}_t$, which is exactly the "coupled gates" LSTM variant.

4.9.5 LSTM vs GRU Comparison

Feature	LSTM	GRU
Number of gates	3 (forget, input, output)	2 (reset, update)
State vectors	2 ($\mathbf{C}_t$, $\mathbf{h}_t$)	1 ($\mathbf{h}_t$)
Parameters per layer	$4n(n+m)+4n$	$3n(n+m)+3n$
Weight matrices	4	3
Training speed	Slower	~25% faster
Long sequences	Slightly better	Slightly worse
Small datasets	Overfits more easily	Better generalization
Dominant usage	NLP (pre-Transformer era)	Speech, time series

Table 4.2: Detailed comparison of LSTM and GRU architectures.

Key Insight:

When to Choose LSTM vs GRU In practice:

- **Choose LSTM** when you have abundant data and compute, and the task requires capturing very long-range dependencies (e.g., document-level language modeling).
- **Choose GRU** when you have limited compute or smaller datasets, or when the sequences are moderately long. GRU often matches LSTM performance with faster training.
- **Choose neither** for modern NLP tasks: use a Transformer. LSTMs and GRUs remain relevant for edge deployment, streaming applications, and domains where Transformers are overkill.

Example:

GRU in PyTorch

```
import torch
import torch.nn as nn

class SentimentGRU(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
        num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
                                      padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)
```

```python

    def forward(self, x):
        # x: (batch, seq_len) of token indices
        emb = self.embedding(x)            # (batch, seq_len, embed_dim)
        _, h_n = self.gru(emb)             # h_n: (1, batch, hidden_dim)
        h_n = h_n.squeeze(0)              # (batch, hidden_dim)
        logits = self.fc(h_n)              # (batch, num_classes)
        return logits

# Example usage
model = SentimentGRU(vocab_size=10000, embed_dim=128,
                     hidden_dim=256, num_classes=2)
x = torch.randint(0, 10000, (32, 50))     # batch of 32, length 50
logits = model(x)                          # (32, 2)
print(f"Output shape: {logits.shape}")
```

4.10 The Complete Picture

Let us step back and survey the landscape we have traversed. The evolution of neural architectures for sequence modeling follows a clear narrative arc, where each innovation addresses a specific limitation of its predecessor.

4.10.1 The Evolution of Sequence Models

Perceptron (single neuron)	→	**MLP / FFN** (multiple layers)	→	**RNN** (adds memory)
		→	**LSTM** (gated memory)	→ **GRU** (simplified gates)
			→	**Transformers** (parallel attention)

4.10.2 Key Equations Reference

Model	Core Equation	Key Property
Perceptron	$y = \sigma(\mathbf{w}^\top \mathbf{x} + b)$	No memory
RNN	$\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b})$	Memory via $\mathbf{h}_{t-1}$
LSTM	$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$	Additive memory
GRU	$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$	Interpolated memory

Table 4.3: Key equations across the evolution of sequence models.

4.10.3 The Three Fundamental Limitations Motivating Transformers

Despite the success of LSTMs and GRUs, three fundamental limitations remain — and no amount of gating can resolve them:

1. **The Encoding Bottleneck.** In encoder-decoder architectures (e.g., for machine translation), the entire source sentence must be compressed into a single fixed-size vector $\mathbf{h}_T$. A 500-word paragraph and a 5-word sentence are both squeezed into the same n-dimensional vector. Critical details are inevitably lost.
2. **No Parallelization.** Because each hidden state $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, the computation at time step t cannot begin until time step $t-1$ has finished. This makes RNNs, LSTMs, and GRUs inherently **sequential**. On modern GPUs with thousands of parallel cores, this means most of the hardware sits idle. Training a deep LSTM on a long sequence is embarrassingly slow compared to what the hardware could theoretically achieve.
3. **Limited Long-Range Memory.** Even with gating mechanisms, LSTMs struggle to connect information across very long distances (hundreds or thousands of tokens). The cell state, though protected, still experiences some degradation over time. More importantly, the model can only attend to the past — it has no mechanism to look ahead (without bidirectional extensions) or to directly connect distant tokens.

Example:

The Transformer Solution (Preview) The Transformer architecture, introduced by Vaswani et al. in the landmark paper "Attention Is All You Need" (2017), addresses all three limitations simultaneously:

1. **No bottleneck**: Instead of compressing everything into one vector, the Transformer allows the decoder to *attend* to every position in the encoder output. The decoder can "look at" any word in the source sentence at any time.
2. **Full parallelization**: Self-attention operates on all positions simultaneously via matrix multiplication. There is no sequential dependency. This enables massive speedups on GPUs.
3. **Direct long-range connections**: In self-attention, every token can attend to every other token in the sequence, regardless of distance. The path length between any two tokens is $O(1)$, compared to $O(T)$ in an RNN.

The Transformer achieves all of this by replacing recurrence entirely with **attention mechanisms**. This is the subject of Chapter 5.

4.11 Conclusion

This chapter has traced the evolution from static word embeddings to contextual embeddings through the lens of recurrent neural networks. The key ideas are:

1. **Static embeddings are insufficient**: Word2Vec and GloVe assign one fixed vector per word, regardless of context. The same word can have radically different meanings in different contexts.
2. **RNNs introduce memory**: By maintaining a hidden state $\mathbf{h}_t$ that is updated at each time step, RNNs create **contextual embeddings** — representations that depend on the entire preceding sequence.
3. **Vanilla RNNs fail on long sequences**: The vanishing gradient problem causes gradients to decay exponentially through time, making it impossible to learn long-range dependencies.
4. **LSTMs solve vanishing gradients**: The cell state $\mathbf{C}_t$, updated by **addition** rather than multiplication, provides a gradient "highway" that allows information to persist across hundreds of time steps. Three gates (forget, input, output) control the flow of information.
5. **GRUs offer a simpler alternative**: By merging the cell and hidden states and using only two gates (reset, update), GRUs achieve comparable performance with fewer parameters

and faster training.

6. **Sequential processing is the bottleneck**: All recurrent architectures — RNN, LSTM, GRU — share a fundamental limitation: they process tokens one at a time. This prevents parallelization and limits scalability.

The stage is now set for the Transformer. In Chapter 5, we will see how the **attention mechanism** eliminates recurrence entirely, enabling parallel computation, direct long-range connections, and the massive scale that powers modern language models like GPT-4, LLaMA, and Claude.